Universidade Federal de Viçosa

Campus

Rio Paranaíba

Leonardo Padovani Manzato – 9862

Projeto de análise de algoritmos de ordenação

Rio Paranaíba – MG

2026

Universidade Federal de Viçosa

Campus

Rio Paranaíba

Leonardo Padovani Manzato – 9862

Projeto de análise de algoritmos de ordenação

Trabalho apresentado para obtenção de créditos na disciplina SIN213 – Projeto de Algoritmo da Universidade Federal de Viçosa – Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba – MG

2026

1 RESUMO

Os algoritmos de ordenação apresentam grande importância no estudo teórico de algoritmos, bem como em aplicações práticas do cotidiano. Esses algoritmos são responsáveis por organizar conjuntos de elementos de acordo com determinado critério, facilitando a busca, a visualização e o processamento dos dados. Diante da existência de diferentes métodos de ordenação, torna-se importante compreender seu funcionamento e analisar seus custos para identificar suas características e limitações.

Neste trabalho, são analisados seis algoritmos de ordenação: *Insertion Sort*, *Bubble Sort*, *Selection Sort*, *Shell Sort*, *Merge Sort* e *Quick Sort*. Cada algoritmo apresenta uma estratégia diferente para organizar os elementos de um vetor. Enquanto o *Insertion Sort* realiza inserções progressivas, o *Bubble Sort* utiliza trocas entre elementos adjacentes, o *Selection Sort* seleciona sucessivamente o menor elemento, o *Shell Sort* utiliza intervalos (*gaps*) para realizar ordenações parciais, o *Merge Sort* emprega a estratégia de divisão e conquista com intercalação de subvetores e o *Quick Sort* utiliza divisão e conquista com particionamento em torno de um pivô. Para o *Quick Sort*, foram implementadas duas versões: uma que utiliza o primeiro elemento como pivô e outra que define o pivô por meio da média aritmética dos elementos da partição.

Para cada algoritmo, são apresentados sua implementação, um teste de mesa manual e uma análise de complexidade, considerando os diferentes casos de execução. Também são realizados experimentos com entradas ordenadas de forma crescente, decrescente e aleatória, utilizando diferentes tamanhos de instância. Os tempos de execução obtidos são apresentados por meio de tabelas e gráficos, permitindo relacionar o comportamento observado experimentalmente com a análise teórica de cada algoritmo, inclusive a influência da escolha do pivô no desempenho do *Quick Sort*.

Dessa forma, o trabalho busca compreender as diferenças entre os algoritmos estudados e analisar como suas características influenciam o desempenho durante a ordenação de diferentes tipos e tamanhos de entrada.

Sumário

1 RESUMO **1**

2 INTRODUÇÃO **5**

3 ALGORITMOS **7**

3.1 Insertion Sort 7

3.2 Bubble Sort 7

3.3 Selection Sort 9

3.4 Shell Sort 10

3.5 Merge Sort 11

3.6 Quick Sort 14

4 ANÁLISE DE COMPLEXIDADE **17**

4.1 Insertion Sort 17

4.1.1 Melhor caso – $\Theta(n)$ 17

4.1.2 Pior caso – $\Theta(n^2)$ 17

4.1.3 Caso médio – $\Theta(n^2)$ 18

4.1.4 Resumo das complexidades 19

4.2 Bubble Sort 19

4.2.1 Melhor caso – $\Theta(n^2)$ 19

4.2.2 Pior caso – $\Theta(n^2)$ 20

4.2.3 Caso médio – $\Theta(n^2)$ 20

4.2.4 Resumo das complexidades 21

4.3 Selection Sort 21

4.3.1 Melhor caso – $\Theta(n^2)$ 22

4.3.2 Pior caso – $\Theta(n^2)$ 22

4.3.3 Caso médio – $\Theta(n^2)$ 23

4.3.4 Resumo das complexidades 24

4.4 Shell Sort 24

4.4.1 Melhor caso 25

4.4.2 Pior caso 25

4.4.3 Caso médio 26

4.4.4 Resumo das complexidades 27
4.5 Merge Sort 27
4.5.1 Melhor caso – $\Theta(n \log n)$ 27
4.5.2 Pior caso – $\Theta(n \log n)$ 28
4.5.3 Caso médio – $\Theta(n \log n)$ 28
4.5.4 Resumo das complexidades 29
4.6 Quick Sort 29
4.6.1 Melhor caso – $\Theta(n \log n)$ 29
4.6.2 Pior caso – $\Theta(n^2)$ 30
4.6.3 Caso médio – $\Theta(n \log n)$ 31
4.6.4 Resumo das complexidades 31

5 TABELAS E GRÁFICOS 32
5.1 Insertion Sort 32
5.2 Bubble Sort 33
5.3 Selection Sort 35
5.4 Shell Sort 36
5.5 Merge Sort 38
5.6 Quick Sort 39

6 COMPARAÇÃO ENTRE OS ALGORITMOS 42
6.1 Entrada crescente 42
6.2 Entrada decrescente 43
6.3 Entrada aleatória 44

7 ATIVIDADES MANUSCRITAS 46
7.1 Insertion Sort 46
7.2 Bubble Sort 48
7.3 Selection Sort 50
7.4 Shell Sort 52
7.5 Merge Sort 54
7.6 Quick Sort 56
7.6.1 Versão 1 (pivô no primeiro elemento) 56
7.6.2 Versão 2 (pivô pela média aritmética) 58

8 CONCLUSÃO **60**

9 REFERÊNCIAS BIBLIOGRÁFICAS **62**

10 Funções **65**

10.1 Calculo do tempo de execução 65

10.2 Execução dos algoritmos 66

2 INTRODUÇÃO

Em várias situações do nosso dia a dia, nos deparamos com a necessidade de trabalhar com dados e informações ordenados de forma correta, como, por exemplo, ao procurar um contato na lista telefônica. Seria difícil localizar um nome caso os contatos não estivessem organizados em ordem alfabética. Dessa forma, fica fácil notar que as atividades que envolvem algum método de ordenação estão muito presentes na computação.

Os algoritmos de ordenação são responsáveis por organizar um conjunto de elementos de acordo com determinado critério, como ordem crescente, decrescente ou alfabética. Essa organização facilita a busca, a visualização e o processamento dos dados, contribuindo para a eficiência de diversas aplicações.

Existem diferentes algoritmos de ordenação, cada um apresentando características, estratégias e custos computacionais próprios. Entre eles, destacam-se os algoritmos *Insertion Sort*, *Bubble Sort*, *Selection Sort*, *Shell Sort*, *Merge Sort* e *Quick Sort*. Apesar de possuírem o mesmo objetivo, esses algoritmos utilizam diferentes métodos para organizar os elementos de um vetor.

O *Insertion Sort*, também conhecido como ordenação por inserção, constrói progressivamente uma parte ordenada do vetor, inserindo cada novo elemento em sua posição correta. O *Bubble Sort* realiza comparações entre elementos adjacentes e troca suas posições quando estão fora de ordem. Já o *Selection Sort* procura o menor elemento da parte ainda não ordenada e o posiciona na posição correspondente. O *Shell Sort* utiliza intervalos, chamados de *gaps*, para realizar ordenações parciais. O *Merge Sort* utiliza a estratégia de divisão e conquista, dividindo o vetor em partes menores e intercalando os resultados até obter a ordenação completa. Por sua vez, o *Quick Sort* também adota a estratégia de divisão e conquista, mas realiza o particionamento do vetor em torno de um elemento escolhido, chamado de pivô, de modo que os elementos menores fiquem de um lado e os maiores do outro, repetindo o processo recursivamente em cada partição.

No *Quick Sort*, a escolha do pivô influencia diretamente o desempenho do algoritmo. Por esse motivo, foram implementadas duas versões: na primeira, o pivô é o primeiro elemento do vetor (ou da partição); na segunda, o pivô é definido a partir da média aritmética dos elementos da partição. Essa diferença pode alterar o balanceamento das partições e, consequentemente, o tempo de execução, principalmente em entradas já ordenadas ou em ordem decrescente.

A análise da complexidade desses algoritmos é importante para compreender como o tempo de execução varia de acordo com o tamanho e a organização inicial da entrada. Dependendo do algoritmo e do caso analisado, podem existir diferenças significativas entre o melhor caso, o caso médio e o pior caso. No *Shell Sort*, além do tamanho da entrada, o comportamento também está relacionado à sequência de *gaps* utilizada. Já no *Quick Sort*, o desempenho depende da qualidade da escolha do pivô, uma vez que partições desbalanceadas levam ao pior caso, enquanto partições equilibradas favorecem o melhor caso e o caso médio.

Neste trabalho, serão apresentados o funcionamento e a implementação dos algoritmos *Insertion Sort*, *Bubble Sort*, *Selection Sort*, *Shell Sort*, *Merge Sort* e *Quick Sort* (nas versões com pivô no primeiro elemento e com pivô pela média aritmética), juntamente com testes de mesa e análises de complexidade. Também serão realizados experimentos utilizando entradas crescentes, decrescentes e aleatórias, considerando diferentes instâncias.

3 ALGORITMOS

3.1 Insertion Sort

O Insertion Sort é um algoritmo de ordenação baseado na comparação e inserção dos elementos em suas posições corretas. Seu funcionamento é semelhante ao ato de organizar cartas de baralho: os elementos são analisados um por vez e cada novo elemento é inserido na posição adequada da parte que já está ordenada.

Inicialmente, considera-se que o primeiro elemento do vetor já está ordenado. Em seguida, o algoritmo seleciona o próximo elemento, chamado de chave, e compara seu valor com os elementos anteriores. Enquanto esses elementos forem maiores que a chave, eles são deslocados uma posição para a direita. Ao final, a chave é inserida na posição correta.

Listing 1: Implementação do Insertion Sort

```
void insertionSort(int* vetor, int tamanho) {
    for (int i = 1; i < tamanho; i++) {

        int chave = vetor[i];
        int j = i - 1;

        while (j >= 0 && vetor[j] > chave) {
            vetor[j + 1] = vetor[j];
            j--;
        }

        vetor[j + 1] = chave;
    }
}
```

3.2 Bubble Sort

O Bubble Sort é um algoritmo de ordenação baseado na comparação e troca de elementos adjacentes. Seu funcionamento consiste em percorrer o vetor várias vezes, comparando elementos vizinhos e realizando uma troca quando eles estão fora da ordem desejada.

Dessa forma, os maiores elementos vão sendo deslocados gradualmente para o final do vetor, de maneira semelhante ao movimento de bolhas subindo em um líquido.

A cada passagem pelo vetor, os elementos adjacentes são comparados. Caso o elemento da esquerda seja maior que o elemento da direita, suas posições são trocadas. Após cada passagem, pelo menos um dos maiores elementos ainda não ordenados é colocado em sua posição definitiva no final do vetor. O processo continua até que todos os elementos estejam ordenados.

Listing 2: Implementação do Bubble Sort

```
void bubbleSort(int* vetor, int tamanho) {
    for (int i = 0; i < tamanho - 1; i++) {
        bool trocou = false;

        for (int j = 0; j < tamanho - i - 1; j++) {
            if (vetor[j] > vetor[j + 1]) {
                int temp = vetor[j];
                vetor[j] = vetor[j + 1];
                vetor[j + 1] = temp;
                trocou = true;
            }
        }
        if (!trocou)
            break;
    }
}
```

No primeiro laço, a variável i representa a quantidade de passagens já realizadas pelo vetor. O segundo laço percorre os elementos ainda não ordenados, comparando cada elemento com seu vizinho à direita.

Quando o elemento da esquerda é maior que o elemento da direita, é realizada uma troca entre eles utilizando a variável auxiliar `temp`. Ao final de cada passagem, o maior elemento da parte ainda não ordenada é levado para o final do vetor. Por isso, o limite do segundo laço diminui a cada nova passagem.

3.3 Selection Sort

O Selection Sort é um algoritmo de ordenação baseado na seleção sucessiva do menor elemento do vetor. Seu funcionamento consiste em dividir mentalmente o vetor em duas partes: uma parte inicialmente desordenada e uma parte que já foi ordenada. A cada iteração, o algoritmo procura o menor elemento da parte desordenada e o coloca na posição correspondente da parte ordenada.

Inicialmente, considera-se que nenhum elemento está ordenado. Durante cada iteração, o algoritmo percorre os elementos restantes do vetor e identifica aquele que possui o menor valor. Após encontrar esse elemento, ele é trocado com o primeiro elemento da parte ainda desordenada. Dessa forma, a parte ordenada cresce progressivamente até que todos os elementos estejam em suas posições corretas.

Listing 3: Implementação do Selection Sort

```
void selectionSort(int* vetor, int tamanho) {
    for (int i = 0; i < tamanho - 1; i++) {
        int menor = i;

        for (int j = i + 1; j < tamanho; j++) {
            if (vetor[j] < vetor[menor]) {
                menor = j;
            }
        }

        int temp = vetor[i];
        vetor[i] = vetor[menor];
        vetor[menor] = temp;
    }
}
```

No primeiro laço, a variável i representa o início da parte ainda não ordenada do vetor. A variável `menor` armazena a posição do menor elemento encontrado durante a busca.

O segundo laço percorre os elementos que ainda não foram ordenados, comparando seus valores com o elemento armazenado na posição `menor`. Quando um elemento menor

é encontrado, sua posição passa a ser armazenada nessa variável.

Ao final de cada iteração, o menor elemento encontrado é trocado com o elemento que ocupa a posição i. Assim, esse elemento passa a fazer parte da região ordenada do vetor. O processo é repetido até que restem apenas dois elementos ou até que todo o vetor esteja ordenado.

3.4 Shell Sort

O Shell Sort é um algoritmo de ordenação baseado no Insertion Sort, porém utiliza comparações entre elementos que estão separados por determinados intervalos, chamados de *gaps*. Essa estratégia permite reduzir gradualmente a desordem do vetor antes da etapa final de ordenação.

Inicialmente, é definido um valor de *gap* que determina a distância entre os elementos que serão comparados. O vetor é então percorrido considerando essa distância, realizando uma ordenação por inserção entre os elementos pertencentes aos diferentes grupos formados. Após cada etapa, o valor do *gap* é reduzido utilizando um divisor, fazendo com que os elementos sejam comparados a distâncias cada vez menores.

O processo continua até que o *gap* seja igual a 1. Nesse momento, é realizada uma última ordenação semelhante ao Insertion Sort, porém o vetor já se encontra parcialmente ordenado devido às etapas anteriores, reduzindo bastante a quantidade de deslocamentos necessários.

Listing 4: Implementação do Shell Sort

```
1  void shellSort(int* vetor, int tamanho, int divisor) {
2
3      for (int gap = tamanho / divisor; gap > 0; gap /= divisor) {
4          for (int i = gap; i < tamanho; i++) {
5
6              int temp = vetor[i];
7
8              int j = i;
9
10             while (j >= gap && vetor[j - gap] > temp) {
11
```

```
12                      vetor[j] = vetor[j - gap];
13                      j -= gap;
14                  }
15
16                  vetor[j] = temp;
17              }
18          }
19      }
```

Na primeira parte do algoritmo, o valor inicial do *gap* é obtido pela divisão do tamanho do vetor pelo divisor informado. A cada nova etapa, o *gap* é novamente dividido pelo mesmo divisor. Dessa forma, o intervalo entre os elementos comparados diminui progressivamente.

O segundo laço percorre o vetor a partir da posição correspondente ao *gap*. Para cada elemento, a variável `temp` armazena seu valor, enquanto o terceiro laço desloca os elementos maiores que `temp` para a posição seguinte dentro do intervalo definido pelo *gap*.

Quando não existem mais elementos maiores que `temp` na sequência considerada, o valor é inserido em sua posição correta. Quando o *gap* chega a 1, o algoritmo realiza uma ordenação por inserção considerando elementos consecutivos. Como as etapas anteriores já reduziram parte da desordem do vetor, essa última etapa tende a exigir menos deslocamentos.

Inicialmente, pretendia-se permitir a alteração do divisor utilizado na redução dos *gaps* durante a execução do algoritmo, com o objetivo de comparar o desempenho do Shell Sort utilizando diferentes sequências de *gaps*. Entretanto, devido a dificuldades na implementação dessa alteração, foi adotado o divisor padrão igual a 2 para os experimentos realizados neste trabalho. Dessa forma, a sequência de *gaps* é obtida pela divisão sucessiva do tamanho do vetor por 2 até que o *gap* seja igual a 1.

3.5 Merge Sort

O Merge Sort é um algoritmo de ordenação baseado na estratégia de divisão e conquista. Seu funcionamento consiste em dividir o vetor em partes menores, ordenar essas partes recursivamente e, em seguida, combinar os resultados para formar um único vetor ordenado.

Inicialmente, o vetor é dividido em duas metades. Esse processo continua até que cada parte contenha apenas um elemento, situação em que ela já está ordenada. Em seguida, a função *merge* combina as duas metades, comparando seus elementos e colocando-os em ordem crescente no vetor original.

A função *mergeSort* é responsável pelas divisões recursivas. Ela calcula o índice do meio do vetor, ordena a metade esquerda, ordena a metade direita e chama a função *merge* para intercalar os resultados. A função *merge*, por sua vez, utiliza dois vetores auxiliares para armazenar temporariamente os elementos das duas metades durante a intercalação.

Listing 5: Implementação do Merge Sort

```
void merge(int* vetor, int esq, int meio, int dir) {
    int n1 = meio - esq + 1;
    int n2 = dir - meio;

    int* E = new int[n1];
    int* D = new int[n2];

    for (int i = 0; i < n1; i++)
        E[i] = vetor[esq + i];

    for (int j = 0; j < n2; j++)
        D[j] = vetor[meio + 1 + j];

    int i = 0, j = 0;
    int k = esq;

    while (i < n1 && j < n2) {
        if (E[i] <= D[j]) {
            vetor[k] = E[i];
            i++;
        } else {
            vetor[k] = D[j];
            j++;
        }
        k++;
```

```
26      }
27
28      while (i < n1) {
29          vetor[k] = E[i];
30          i++;
31          k++;
32      }
33
34      while (j < n2) {
35          vetor[k] = D[j];
36          j++;
37          k++;
38      }
39
40      delete[] E;
41      delete[] D;
42  }
43
44  void mergeSort(int* vetor, int tamanho,
45                 int esq = 0, int dir = -1) {
46      if (dir == -1)
47          dir = tamanho - 1;
48
49      if (esq >= dir)
50          return;
51
52      int meio = esq + (dir - esq) / 2;
53
54      mergeSort(vetor, tamanho, esq, meio);
55      mergeSort(vetor, tamanho, meio + 1, dir);
56
57      merge(vetor, esq, meio, dir);
58  }
```

Ao utilizar a função *mergeSort*, basta informar o vetor e seu tamanho. Os parâ-

metros *esq* e *dir* possuem valores padrão, permitindo que a primeira chamada considere todo o vetor. Nas chamadas recursivas, esses parâmetros delimitam as partes que serão ordenadas.

O processo de divisão termina quando o intervalo possui zero ou um elemento, condição identificada por *esq* maior ou igual a *dir*. Após as chamadas recursivas, as partes ordenadas são intercaladas até que todo o vetor esteja organizado em ordem crescente.

3.6 Quick Sort

O *Quick Sort* é um algoritmo de ordenação baseado na estratégia de divisão e conquista. Sua ideia principal é escolher um elemento do vetor, chamado de pivô, e reorganizar os demais elementos de modo que os valores menores que o pivô fiquem de um lado e os valores maiores fiquem do outro.

Após esse particionamento, o algoritmo é aplicado recursivamente às duas partes formadas, até que cada partição possua no máximo um elemento. Diferentemente do *Merge Sort*, não é necessário um vetor auxiliar para intercalar os resultados, ja que a ordenação ocorre no vetor.

Foram implementadas duas versões do algoritmo, que diferem apenas na função de particionamento utilizada, mais especificamente na escolha do pivô. Na versão 1 (`particao_v1`), o pivô é o primeiro elemento da partição, obtido diretamente em `vetor[esq]`, sem custo adicional. Na versão 2 (`particao_v2`), o pivô é a média aritmética dos elementos da partição, calculada a cada chamada ao percorrer todos os elementos entre `esq` e `dir`, somá-los e dividir o resultado pela quantidade de elementos. Como a média nem sempre corresponde a um elemento do vetor, nessa versão o particionamento é feito por comparação com o valor da média, e não com uma posição específica. Fora isso, as duas funções são idênticas.

Listing 6: Implementação do Quick Sort

```
void quickSort(int* vetor, int tamanho, int versao) {

    auto ordena = [&](auto&& self, int esq, int dir) -> void {
        while (esq < dir) {
            int i, j;

            if (versao == 1)
                particao_v1(vetor, esq, dir, i, j);
            else if (versao == 2)
                particao_v2(vetor, esq, dir, i, j);

            int tamEsq = j - esq + 1;
            int tamDir = dir - i + 1;

            if (tamEsq < tamDir) {
                if (esq < j)
                    self(self, esq, j);

                esq = i;
            } else {
                if (i < dir)
                    self(self, i, dir);

                dir = j;
            }
        }
    };

    if (tamanho > 1 && (versao == 1 || versao == 2))
        ordena(ordena, 0, tamanho - 1);
}
```

Nas duas funções de particionamento, os índices `i` e `j` começam nas extremidades da partição e se aproximam até se cruzarem. O índice `i` avança enquanto encontra elementos

menores que o pivô, e o índice `j` volta enquanto encontra elementos maiores que o pivô. Quando ambos param, e `i` ainda não ultrapassou `j`, os elementos são trocados de posição e os dois índices continuam.

No final do particionamento, o índice `j` indica o fim da partição da esquerda e o índice `i` indica o início da partição da direita, assim, os elementos à esquerda são menores ou iguais ao pivô e os elementos à direita são maiores ou iguais a ele. A diferença entre as versões está, portanto, no custo e na qualidade da escolha do pivô: a versão 1 não exige nenhuma operação extra, enquanto a versão 2 acrescenta um custo linear a cada particionamento, em troca de uma divisão potencialmente mais equilibrada.

A função `quickSort` recebe o vetor, seu tamanho e a versão desejada. Uma função lambda recursiva (`ordena`) recebe os limites `esq` e `dir` da partição atual, chama a função de particionamento correspondente à versão escolhida e calcula o tamanho das duas partições geradas, `tamEsq` e `tamDir`.

Para limitar a profundidade da recursão, a chamada recursiva é feita apenas sobre a menor das duas partições, enquanto a maior é tratada pelo próprio laço `while`, ajustando os limites `esq` ou `dir`. O processo termina quando a partição possui no máximo um elemento.

A escolha do pivô influencia diretamente o balanceamento das partições. Na versão 1, vetores já ordenados de forma crescente ou decrescente tendem a gerar partições muito desiguais, aproximando o algoritmo do pior caso. Na versão 2, a média aritmética tende a dividir melhor os valores da partição, porém exige a soma dos elementos a cada chamada, o que acrescenta um custo adicional. Esses efeitos serão observados nos experimentos apresentados neste trabalho.

4 ANÁLISE DE COMPLEXIDADE

4.1 Insertion Sort

O algoritmo Insertion Sort apresenta diferentes comportamentos de desempenho dependendo da organização inicial dos elementos do vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.1.1 Melhor caso – $\Theta(n)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Nessa situação, para cada elemento analisado, o algoritmo realiza apenas uma comparação com o elemento anterior e verifica que não é necessário deslocar nenhum elemento.

Considerando um vetor com n elementos, o laço externo é executado $n-1$ vezes. Em cada iteração, é realizada uma quantidade constante de operações, como atribuições e comparações.

Assim, o número total de operações é proporcional a:

$$T(n) = c(n-1) + d \tag{1}$$

em que c e d são constantes. Desconsiderando os termos constantes e os de menor crescimento, temos:

$$T(n) = \Theta(n) \tag{2}$$

Portanto, o melhor caso do Insertion Sort possui complexidade linear.

4.1.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está ordenado de forma decrescente e deseja-se ordená-lo de forma crescente. Nesse cenário, cada novo elemento precisa ser comparado com todos os elementos anteriores e deslocado até o início da parte já ordenada.

A quantidade total de deslocamentos pode ser representada pela soma:

$$T(n) = 1 + 2 + 3 + \cdots + (n-1) \tag{3}$$

Essa soma corresponde à soma dos termos de uma progressão aritmética:

$$T(n) = \frac{n(n-1)}{2} \tag{4}$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \tag{5}$$

O termo de maior crescimento é n^2. Portanto:

$$T(n) = \Theta(n^2) \tag{6}$$

Assim, o pior caso do Insertion Sort apresenta complexidade quadrática.

4.1.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos aleatoriamente. Nessa situação, o algoritmo realiza, em média, aproximadamente metade dos deslocamentos possíveis para cada elemento inserido.

Temos:

$$T(n) \approx \frac{1}{2}\left(1 + 2 + 3 + \cdots + (n-1)\right) \tag{7}$$

Aplicando a fórmula da soma:

$$T(n) \approx \frac{1}{2} \cdot \frac{n(n-1)}{2} \tag{8}$$

$$T(n) \approx \frac{n^2 - n}{4} \tag{9}$$

Como o termo dominante é n^2, a complexidade assintótica é:

$$T(n) = \Theta(n^2) \tag{10}$$

Portanto, no caso médio, o Insertion Sort também apresenta complexidade quadrática.

4.1.4 Resumo das complexidades

Tabela 1: Complexidades do algoritmo Insertion Sort

Caso	**Complexidade de tempo**
Melhor caso	$\Theta(n)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Insertion Sort é eficiente para vetores pequenos ou parcialmente ordenados, mas apresenta desempenho inferior para grandes conjuntos de dados desordenados.

4.2 Bubble Sort

O algoritmo Bubble Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois realiza comparações entre elementos adjacentes durante sucessivas passagens pelo vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.2.1 Melhor caso – $\Theta(n^2)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Mesmo nessa situação, a implementação utilizada continua realizando todas as comparações previstas pelos laços do algoritmo, embora nenhuma troca seja necessária.

Considerando um vetor com n elementos, na primeira passagem são realizadas $n-1$ comparações. Na segunda passagem, são realizadas $n-2$ comparações, e assim sucessivamente até a última passagem, que realiza apenas uma comparação.

Assim, o número total de comparações pode ser representado pela soma:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{11}$$

Essa soma corresponde à soma dos primeiros $n-1$ números naturais:

$$T(n) = \frac{n(n-1)}{2} \tag{12}$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \tag{13}$$

O termo de maior crescimento é n^2. Portanto:

$$T(n) = \Theta(n^2) \tag{14}$$

Assim, o melhor caso do Bubble Sort apresenta complexidade quadrática.

4.2.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está ordenado de forma decrescente e deseja-se ordená-lo de forma crescente. Nesse cenário, além de realizar todas as comparações previstas pelos laços, o algoritmo também precisa realizar uma grande quantidade de trocas entre os elementos.

A quantidade de comparações realizadas permanece igual à do melhor caso, sendo representada por:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{15}$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n-1)}{2} \tag{16}$$

$$T(n) = \frac{n^2 - n}{2} \tag{17}$$

Além das comparações, são realizadas trocas dos elementos que estão fora de ordem. No pior caso, a quantidade de trocas também cresce quadraticamente.

Como o termo dominante é n^2, temos:

$$T(n) = \Theta(n^2) \tag{18}$$

Portanto, o pior caso do Bubble Sort apresenta complexidade quadrática.

4.2.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos de maneira aleatória. Nessa situação, o algoritmo continua realizando todas as passagens e comparações previstas pelos seus laços.

A quantidade de comparações é dada por:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{19}$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n-1)}{2} \tag{20}$$

$$T(n) = \frac{n^2 - n}{2} \tag{21}$$

Embora a quantidade de trocas varie de acordo com a organização inicial dos elementos, o número de comparações continua sendo proporcional a n^2.

Como o termo dominante é n^2, a complexidade assintótica é:

$$T(n) = \Theta(n^2) \tag{22}$$

Portanto, no caso médio, o Bubble Sort também apresenta complexidade quadrática.

4.2.4 Resumo das complexidades

Tabela 2: Complexidades do algoritmo Bubble Sort

Caso	**Complexidade de tempo**
Melhor caso	$\Theta(n)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Bubble Sort apresenta complexidade quadrática nos três casos analisados. Mesmo quando o vetor já está ordenado, a implementação utilizada continua realizando as comparações das passagens previstas. Por esse motivo, o algoritmo pode apresentar desempenho limitado para grandes conjuntos de dados.

4.3 Selection Sort

O algoritmo Selection Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois a quantidade de comparações realizadas não depende da organização inicial

dos elementos do vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.3.1 Melhor caso – $\Theta(n^2)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Mesmo nessa situação, o algoritmo precisa percorrer a parte ainda não ordenada do vetor para verificar qual é o menor elemento.

Considerando um vetor com n elementos, na primeira iteração são realizadas $n-1$ comparações. Na segunda iteração, são realizadas $n-2$ comparações, e assim sucessivamente até a última iteração, que realiza apenas uma comparação.

A quantidade total de comparações pode ser representada pela soma:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{23}$$

Essa soma corresponde à soma dos primeiros $n-1$ números naturais:

$$T(n) = \frac{n(n-1)}{2} \tag{24}$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \tag{25}$$

O termo de maior crescimento é n^2. Portanto:

$$T(n) = \Theta(n^2) \tag{26}$$

Assim, o melhor caso do Selection Sort apresenta complexidade quadrática.

4.3.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está organizado de forma que seja necessário realizar várias trocas para colocá-lo em ordem crescente. Entretanto, mesmo nesse cenário, a quantidade de comparações realizadas pelo algoritmo permanece a mesma.

Na primeira iteração, são realizadas $n-1$ comparações; na segunda, $n-2$ comparações, e assim sucessivamente. Dessa forma:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{27}$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n-1)}{2} \tag{28}$$

$$T(n) = \frac{n^2 - n}{2} \tag{29}$$

Como o termo dominante é n^2, temos:

$$T(n) = \Theta(n^2) \tag{30}$$

Portanto, o pior caso do Selection Sort apresenta complexidade quadrática.

4.3.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos de maneira aleatória. Nessa situação, assim como nos demais casos, o algoritmo precisa percorrer toda a parte ainda não ordenada do vetor para encontrar o menor elemento.

A quantidade de comparações continua sendo representada por:

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1 \tag{31}$$

Aplicando a fórmula da soma:

$$T(n) = \frac{n(n-1)}{2} \tag{32}$$

$$T(n) = \frac{n^2 - n}{2} \tag{33}$$

Como o termo dominante é n^2, a complexidade assintótica é:

$$T(n) = \Theta(n^2) \tag{34}$$

Portanto, no caso médio, o Selection Sort também apresenta complexidade quadrática.

4.3.4 Resumo das complexidades

Tabela 3: Complexidades do algoritmo Selection Sort

Caso	**Complexidade de tempo**
Melhor caso	$\Theta(n^2)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Selection Sort apresenta complexidade quadrática nos três casos analisados. Diferentemente do Insertion Sort, a organização inicial do vetor não reduz significativamente a quantidade de comparações, pois o algoritmo precisa procurar o menor elemento em toda a parte ainda não ordenada a cada iteração.

4.4 Shell Sort

O algoritmo Shell Sort apresenta um comportamento diferente dos demais algoritmos analisados, pois sua complexidade depende da sequência de *gaps* utilizada. Neste trabalho, o *gap* inicial é obtido pela divisão do tamanho do vetor pelo divisor escolhido e, a cada etapa, é novamente dividido pelo mesmo divisor até atingir o valor 1.

Na implementação utilizada nos experimentos, foi adotado o divisor igual a 2. Dessa forma, para um vetor de tamanho n, a sequência de *gaps* é obtida aproximadamente por:

$$\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \ldots, 1 \tag{35}$$

Por exemplo, para um vetor de tamanho 64, utilizando o divisor 2, a sequência de *gaps* será:

$$32, 16, 8, 4, 2, 1 \tag{36}$$

Já utilizando o divisor 4, a sequência será:

$$16, 4, 1 \tag{37}$$

Assim, um divisor maior tende a reduzir a quantidade de etapas realizadas. Entretanto, isso não significa necessariamente que o algoritmo será mais rápido, pois o desempenho também depende da quantidade de comparações e deslocamentos realizados em cada etapa

e da desordem restante quando o *gap* atinge o valor 1.

4.4.1 Melhor caso

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Nessa situação, os elementos já estão nas posições adequadas, de modo que as etapas de ordenação não precisam realizar deslocamentos significativos.

Em cada etapa, o algoritmo percorre os elementos do vetor, realizando comparações para verificar se a ordem está correta. Considerando uma sequência de *gaps* obtida pela divisão sucessiva por uma constante maior que 1, o número de etapas é proporcional a $\log n$.

Como cada etapa realiza aproximadamente n verificações no vetor, o custo pode ser estimado por:

$$T(n) = O(n \log n) \tag{38}$$

Essa expressão representa uma estimativa para o número de operações nesse cenário, considerando a sequência de *gaps* adotada. A complexidade exata deve levar em conta os detalhes da implementação e a sequência utilizada.

4.4.2 Pior caso

No pior caso, a disposição inicial dos elementos pode exigir uma grande quantidade de comparações e deslocamentos. Embora as etapas com *gaps* maiores reduzam parte da desordem, a etapa final, com *gap* igual a 1, pode exigir um número elevado de operações.

Nessa etapa, o Shell Sort realiza uma ordenação equivalente ao Insertion Sort sobre o vetor inteiro. No pior caso, a quantidade de operações dessa etapa pode ser representada por:

$$T(n) = 1 + 2 + 3 + \cdots + (n-1) \tag{39}$$

Aplicando a fórmula da soma dos termos de uma progressão aritmética, temos:

$$T(n) = \frac{n(n-1)}{2} \tag{40}$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \tag{41}$$

O termo de maior crescimento é n^2. Portanto, a etapa final pode apresentar custo quadrático:

$$T(n) = \Theta(n^2) \tag{42}$$

Entretanto, essa análise considera o custo máximo da etapa com *gap* igual a 1. Para determinar a complexidade de pior caso do Shell Sort completo, é necessário considerar também o efeito das etapas anteriores e a sequência de *gaps* utilizada.

4.4.3 Caso médio

O caso médio considera entradas com diferentes disposições dos elementos, como vetores gerados aleatoriamente. Diferentemente de algoritmos como Insertion Sort, Bubble Sort e Selection Sort, cuja análise pode ser apresentada sem depender de uma sequência de intervalos, o comportamento do Shell Sort depende diretamente da sequência de *gaps* adotada.

Alterar o divisor modifica a quantidade de etapas e os intervalos utilizados nas comparações. Por exemplo, para um vetor de tamanho 64, o divisor 2 produz a sequência:

$$32, 16, 8, 4, 2, 1 \tag{43}$$

Enquanto o divisor 4 produz:

$$16, 4, 1 \tag{44}$$

Apesar de a segunda sequência possuir menos etapas, isso não garante um menor tempo de execução. O desempenho depende de como cada etapa reorganiza os elementos e de quanto trabalho permanece para a etapa final.

Dessa forma, não é adequado atribuir uma única complexidade média ao Shell Sort sem especificar a sequência de *gaps* e as condições consideradas na análise. Neste trabalho, o caso médio será avaliado experimentalmente por meio dos tempos de execução obtidos para vetores com elementos em ordem aleatória.

4.4.4 Resumo das complexidades

Tabela 4: Análise da complexidade do algoritmo Shell Sort

Caso	**Complexidade de tempo**
Melhor caso	Depende da sequência de *gaps*
Caso médio	Depende da sequência de *gaps*
Pior caso	Depende da sequência de *gaps*

Conclui-se que o Shell Sort apresenta uma complexidade que depende da sequência de *gaps* utilizada. O divisor determina como os intervalos são reduzidos e influencia a quantidade de etapas e de operações realizadas durante a ordenação.

Neste trabalho, foi utilizado o divisor 2 nos experimentos, produzindo uma sequência de *gaps* obtida pela divisão sucessiva por 2 até atingir o valor 1. Uma possível extensão dos experimentos seria comparar diferentes divisores, permitindo investigar como a escolha da sequência de *gaps* influencia o tempo de execução do algoritmo.

4.5 Merge Sort

O algoritmo Merge Sort apresenta comportamento semelhante nos casos crescente, decrescente e aleatório, pois divide recursivamente o vetor em duas partes e realiza a intercalação dos elementos em cada nível da recursão.

4.5.1 Melhor caso – $\Theta(n \log n)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Mesmo nessa situação, o algoritmo continua dividindo o vetor em duas metades até atingir intervalos com um único elemento.

Em seguida, as metades são intercaladas. Como a implementação realiza a cópia dos elementos para vetores auxiliares e percorre as partes durante a intercalação, o trabalho realizado em cada nível é proporcional a n.

A quantidade de níveis de divisão é proporcional a $\log_2 n$. Portanto, o custo total pode ser representado por:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \tag{45}$$

em que c representa uma constante associada ao trabalho de intercalação.

Como existem aproximadamente $\log_2 n$ níveis, cada um com custo proporcional a n, temos:

$$T(n) = \Theta(n \log n) \tag{46}$$

Assim, o melhor caso do Merge Sort apresenta complexidade linearítmica.

4.5.2 Pior caso – $\Theta(n \log n)$

O pior caso ocorre quando a intercalação precisa comparar os elementos das duas metades até que uma delas seja completamente percorrida. Isso pode acontecer quando os elementos estão distribuídos de maneira que seja necessário intercalar valores de ambas as metades.

Apesar do aumento do trabalho de comparação em cada intercalação, o algoritmo continua dividindo o vetor em duas partes e processando todos os elementos em cada nível.

Dessa forma, a recorrência permanece:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \tag{47}$$

Como o custo por nível é proporcional a n e existem aproximadamente $\log_2 n$ níveis, temos:

$$T(n) = \Theta(n \log n) \tag{48}$$

Portanto, o pior caso do Merge Sort também apresenta complexidade linearítmica.

4.5.3 Caso médio – $\Theta(n \log n)$

No caso médio, considera-se uma distribuição aleatória dos elementos do vetor. O algoritmo realiza as divisões recursivas da mesma forma e intercala as partes ordenadas até completar a ordenação.

Independentemente da distribuição inicial, todos os elementos precisam ser processados durante as intercalações de cada nível. Assim, o trabalho total continua sendo proporcional a n por nível.

Consequentemente, temos:

$$T(n) = \Theta(n \log n) \tag{49}$$

Portanto, o caso médio do Merge Sort apresenta complexidade linearítmica.

4.5.4 Resumo das complexidades

Tabela 5: Complexidades do algoritmo Merge Sort

Caso	**Complexidade de tempo**
Melhor caso	$\Theta(n \log n)$
Caso médio	$\Theta(n \log n)$
Pior caso	$\Theta(n \log n)$

Conclui-se que o Merge Sort mantém complexidade temporal $\Theta(n \log n)$ nos três casos analisados. Entretanto, sua implementação convencional necessita de memória auxiliar proporcional ao tamanho do vetor.

4.6 Quick Sort

O comportamento do *Quick Sort* depende diretamente do balanceamento das partições geradas a cada particionamento, que por sua vez depende da escolha do pivô. Por esse motivo, diferentemente do *Merge Sort*, os casos crescente, decrescente e aleatório podem apresentar custos diferentes. Neste trabalho, a análise considera as duas versões implementadas: a versão 1, com pivô no primeiro elemento, e a versão 2, com pivô dado pela média aritmética da partição.

Em ambas as versões, o particionamento percorre os elementos da partição uma única vez com os índices `i` e `j`, realizando trabalho proporcional ao tamanho da partição. Na versão 2, o cálculo da média também percorre a partição uma vez, o que mantém o custo de cada particionamento proporcional a n.

4.6.1 Melhor caso – $\Theta(n \log n)$

O melhor caso ocorre quando, a cada particionamento, o pivô divide a partição em duas partes de tamanhos aproximadamente iguais. Nessa situação, cada chamada gera duas subpartições com cerca de $n/2$ elementos, e o custo total pode ser representado por:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn \tag{50}$$

em que cn representa o custo do particionamento, proporcional ao tamanho da partição.

Como existem aproximadamente $\log_2 n$ níveis de divisão, cada um com custo total proporcional a n, temos:

$$T(n) = \Theta(n \log n) \tag{51}$$

Na versão 1, isso ocorre quando o primeiro elemento de cada partição é, aproximadamente, a mediana dos seus elementos. Na versão 2, ocorre quando a média aritmética está próxima da mediana, como em vetores cujos valores são distribuídos de forma uniforme, mesmo que estejam ordenados. Nesse caso, o cálculo da média apenas soma outro custo linear ao particionamento, sem alterar a ordem de crescimento.

4.6.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o pivô divide a partição de forma extremamente desbalanceada, gerando uma partição com $n - 1$ elementos e outra vazia (ou com um único elemento). Nesse caso, a recorrência é:

$$T(n) = T(n-1) + cn \tag{52}$$

Expandindo a recorrência, obtém-se:

$$T(n) = c\sum_{k=1}^{n} k = c\,\frac{n(n+1)}{2} = \Theta(n^2) \tag{53}$$

Na versão 1, esse comportamento ocorre quando o primeiro elemento da partição é sempre o menor ou o maior valor dela, o que acontece tanto em vetores já ordenados de forma crescente quanto em vetores ordenados de forma decrescente. Em vetores crescentes, o pivô é o menor elemento e a partição da esquerda fica vazia; em vetores decrescentes, o pivô é o maior elemento e a partição da direita fica com um único elemento.

Na versão 2, essas entradas deixam de ser o pior caso, pois a média aritmética de valores distribuídos uniformemente tende a ficar próxima do centro da partição. Entretanto, o

pior caso ainda é possível quando os valores são muito discrepantes, como em um vetor cujos elementos crescem de forma exponencial. Nesse caso, a média é maior que quase todos os elementos, e cada particionamento separa apenas o maior valor, gerando a mesma recorrência acima. Assim, o pior caso da versão 2 depende da distribuição dos valores, e não apenas da ordem em que eles aparecem.

4.6.3 Caso médio – $\Theta(n \log n)$

No caso médio, considera-se uma distribuição aleatória dos elementos do vetor. Nessa situação, a posição final do pivô pode ser qualquer uma das n posições com a mesma probabilidade, e a recorrência torna-se:

$$T(n) = \frac{1}{n} \sum_{k=0}^{n-1} [T(k) + T(n-k-1)] + cn \tag{54}$$

A solução dessa recorrência é aproximadamente $2n \ln n$, o que resulta em:

$$T(n) = \Theta(n \log n) \tag{55}$$

Mesmo quando as partições não são perfeitamente equilibradas, divisões com proporção constante (por exemplo, 1/10 e 9/10) ainda produzem profundidade logarítmica. Por esse motivo, o caso médio se aproxima do melhor caso. Na versão 2, como a média tende a ficar próxima da mediana em entradas aleatórias com valores uniformes, o comportamento esperado é equivalente, porém com o custo adicional da soma dos elementos em cada particionamento.

4.6.4 Resumo das complexidades

Tabela 6: Complexidades do algoritmo Quick Sort

Caso	**Versão 1 (primeiro elemento)**	**Versão 2 (média)**
Melhor caso	$\Theta(n \log n)$	$\Theta(n \log n)$
Caso médio	$\Theta(n \log n)$	$\Theta(n \log n)$
Pior caso	$\Theta(n^2)$	$\Theta(n^2)$

Conclui-se que o *Quick Sort* apresenta complexidade $\Theta(n \log n)$ nos casos melhor e médio, e $\Theta(n^2)$ no pior caso, em ambas as versões.

5 TABELAS E GRÁFICOS

5.1 Insertion Sort

Resultados dos experimentos — Insertion Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	0.0	3e-06	3e-06
1000.0	1e-06	0.000239	0.000166
10000.0	7e-06	0.048499	0.0233
100000.0	6.8e-05	4.964015	2.45468
1000000.0	0.001696	535.4135	272.5905

Figura 1: Tabela de tempo de execução do algoritmo Insertion Sort

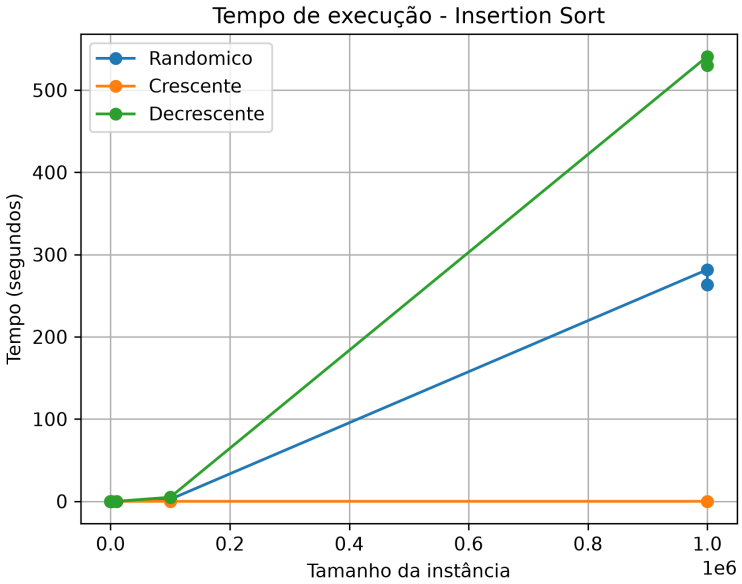


Figura 2: Gráfico de tempo de execução do algoritmo Insertion Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Insertion Sort apresenta melhor desempenho para instâncias ordenadas de forma crescente, uma vez que, nesse caso, praticamente não são necessárias movimentações dos elementos.

Em contrapartida, as instâncias ordenadas de forma decrescente apresentam os maiores tempos de execução. Isso ocorre porque cada novo elemento precisa ser comparado e deslocado por uma grande quantidade de posições, caracterizando o pior caso do algoritmo, cuja complexidade é $\Theta(n^2)$.

As instâncias aleatórias apresentam desempenho intermediário, pois a quantidade de deslocamentos depende da disposição inicial dos elementos.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução, principalmente nos casos aleatório e decrescente. Para a instância de tamanho $n = 1.000.000$, o tempo registrado foi aproximadamente 272,59 segundos para a entrada aleatória e 535,41 segundos para a entrada decrescente. Já para a entrada crescente, o tempo foi de aproximadamente 0,001696 segundos.

Esses resultados confirmam o comportamento teórico esperado para o algoritmo Insertion Sort.

5.2 Bubble Sort

Resultados dos experimentos — Bubble Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	0.0	1.4e-05	3e-05
1000.0	1e-06	0.000962	0.001032
10000.0	8e-06	0.106261	0.094531
100000.0	0.0001	10.2698	17.7803
1000000.0	0.000797	1156.37	1954.39

Figura 3: Tabela de tempo de execução do algoritmo Bubble Sort

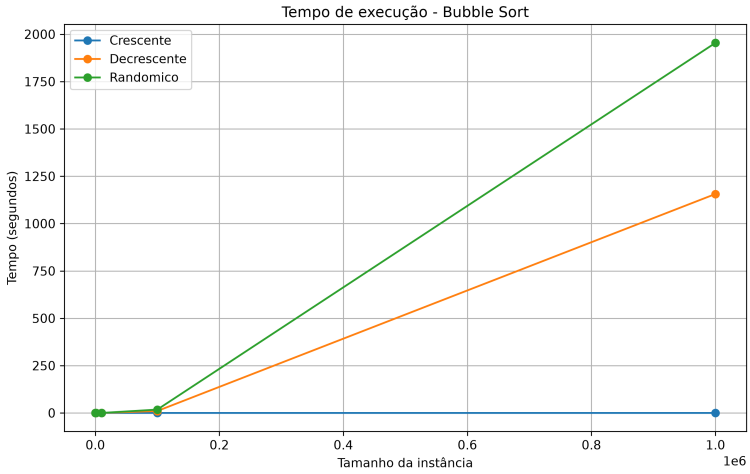


Figura 4: Gráfico de tempo de execução do algoritmo Bubble Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Bubble Sort apresenta comportamentos diferentes de acordo com a organização inicial dos elementos, devido à utilização da variável `trocou`, que permite interromper a execução quando nenhuma troca é realizada durante uma passagem.

Nas instâncias ordenadas de forma crescente, nenhuma troca é realizada durante a primeira passagem. Dessa forma, a variável `trocou` permanece com valor `false` e o algoritmo encerra sua execução por meio do comando `break`, fazendo com que o melhor caso apresente complexidade $\Theta(n)$.

Nas instâncias ordenadas de forma decrescente, além das comparações, também são necessárias diversas trocas para colocar os elementos em suas posições corretas. Como ocorre pelo menos uma troca durante cada passagem, o comando `break` não é executado antes do término das passagens previstas, caracterizando o pior caso do algoritmo, cuja complexidade é $\Theta(n^2)$.

As instâncias aleatórias apresentam uma quantidade intermediária de trocas, pois o número de elementos que precisam ser deslocados depende da organização inicial do vetor. Entretanto, o algoritmo normalmente realiza diversas passagens pelo vetor, fazendo com que o caso médio apresente complexidade $\Theta(n^2)$.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução, principalmente nas entradas aleatórias e decrescentes. Nas entradas crescentes, o algoritmo pode encerrar a execução após a primeira passagem, devido à ausência de trocas.

Esses resultados apresentam comportamento compatível com a análise de complexi-

dade do algoritmo Bubble Sort, considerando a otimização utilizada na implementação por meio da variável `trocou`.

5.3 Selection Sort

Resultados dos experimentos — Selection Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	1e-06	1e-06
100.0	8e-06	1.1e-05	1e-05
1000.0	0.000397	0.000403	0.00041
10000.0	0.035705	0.036678	0.035624
100000.0	3.77752	3.91593	3.70861
1000000.0	403.486	412.212	392.715

Figura 5: Tabela de tempo de execução do algoritmo Selection Sort

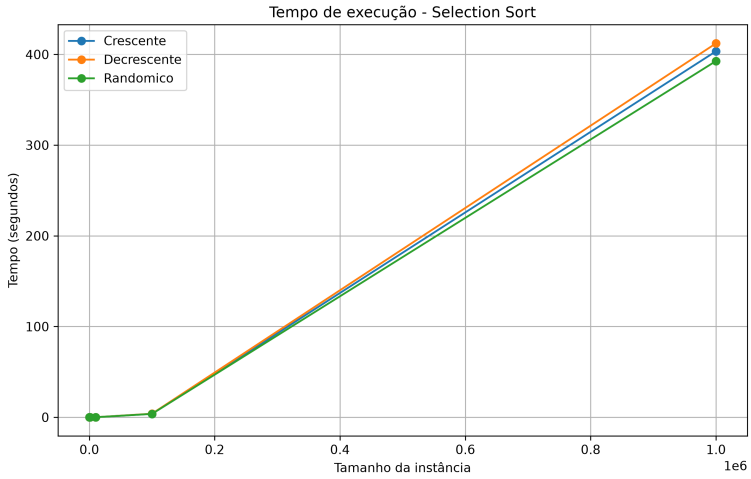


Figura 6: Gráfico de tempo de execução do algoritmo Selection Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Selection Sort apresenta comportamento semelhante nos diferentes tipos de entrada, pois a quantidade de comparações realizadas não depende da organização inicial dos elementos.

Nas instâncias ordenadas de forma crescente, o algoritmo ainda precisa percorrer a parte não ordenada do vetor para encontrar o menor elemento. Dessa forma, mesmo

quando os elementos já estão organizados, são realizadas aproximadamente as mesmas comparações previstas para os demais tipos de entrada.

Nas instâncias ordenadas de forma decrescente, o algoritmo também realiza a mesma quantidade de comparações, embora a posição do menor elemento encontrado possa exigir mais trocas para colocá-lo na posição correta.

As instâncias aleatórias apresentam comportamento semelhante, pois, independentemente da disposição inicial dos elementos, o algoritmo continua procurando o menor elemento em toda a parte ainda não ordenada do vetor.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução. Esse comportamento está de acordo com a análise teórica, segundo a qual o Selection Sort apresenta complexidade $\Theta(n^2)$ no melhor caso, caso médio e pior caso.

Assim, os resultados experimentais apresentam comportamento compatível com a análise de complexidade do algoritmo Selection Sort.

5.4 Shell Sort

Resultados dos experimentos — Shell Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	1e-06	2e-06	5e-06
1000.0	1.2e-05	2e-05	7.8e-05
10000.0	0.000171	0.000275	0.001253
100000.0	0.002235	0.003338	0.016872
1000000.0	0.02662	0.041579	0.214588

Figura 7: Tabela de tempo de execução do algoritmo Shell Sort

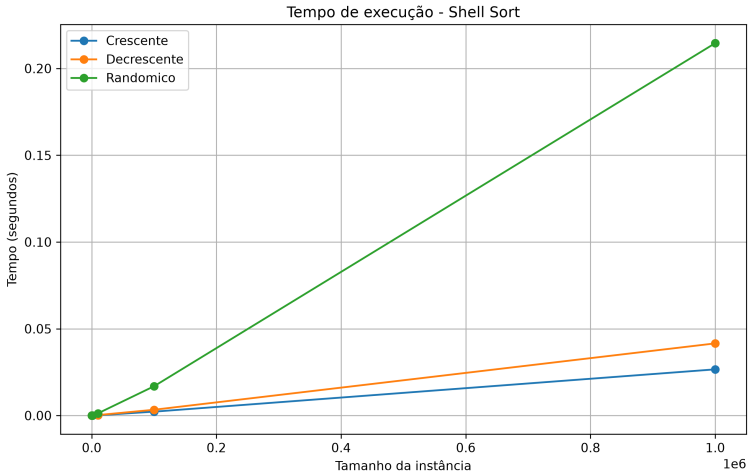


Figura 8: Gráfico de tempo de execução do algoritmo Shell Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. O algoritmo Shell Sort apresenta um comportamento diferente dos demais algoritmos analisados, pois seu desempenho é influenciado pela sequência de *gaps* utilizada. Nos experimentos deste trabalho, foi utilizado o divisor 2, produzindo uma sequência de *gaps* obtida pela divisão sucessiva do tamanho da entrada por 2 até atingir o valor 1.

Nas instâncias ordenadas de forma crescente, os elementos já se encontram em suas posições relativas para os diferentes *gaps*, reduzindo a quantidade de deslocamentos necessários durante a execução.

Nas instâncias ordenadas de forma decrescente, são necessários mais deslocamentos para reorganizar os elementos. Entretanto, os *gaps* maiores permitem que elementos que estão distantes sejam movimentados antes da etapa final, reduzindo parte da desordem do vetor.

As instâncias aleatórias apresentam comportamento intermediário, pois a quantidade de comparações e deslocamentos depende da disposição inicial dos elementos. O uso dos diferentes *gaps* permite realizar movimentações entre elementos distantes antes da ordenação final com *gap* igual a 1.

A partir dos dados, também é possível observar a influência do aumento do tamanho da entrada no tempo de execução. O crescimento observado pode ser relacionado à análise teórica do algoritmo, que depende da sequência de *gaps* utilizada. Para a sequência adotada neste trabalho, o pior caso apresenta comportamento de ordem $\Theta(n^2)$, enquanto o melhor caso apresenta comportamento de ordem $\Theta(n \log n)$.

Os resultados experimentais permitem, portanto, relacionar o comportamento obser-

vado na tabela e no gráfico com a estratégia utilizada pelo Shell Sort e com a influência da sequência de *gaps* em seu desempenho.

5.5 Merge Sort

Resultados dos experimentos — Merge Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	1.5e-05	3e-06	2e-06
100.0	3.4e-05	1.4e-05	1.5e-05
1000.0	0.000116	0.000117	0.000143
10000.0	0.001013	0.001043	0.001477
100000.0	0.010097	0.010023	0.017619
1000000.0	0.114392	0.109827	0.190879

Figura 9: Tabela de tempo de execução do algoritmo Merge Sort

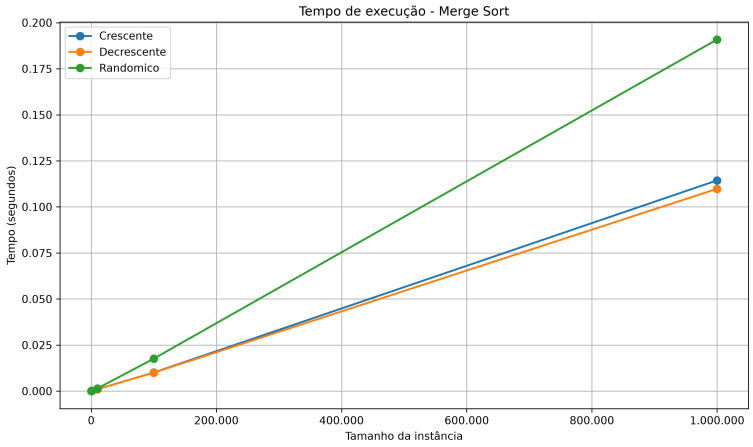


Figura 10: Gráfico de tempo de execução do algoritmo Merge Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observa-se que o algoritmo Merge Sort apresenta comportamento semelhante para as entradas crescentes, decrescentes e aleatórias, pois sua estratégia de divisão e conquista não depende significativamente da organização inicial dos elementos.

Durante a execução, o vetor é dividido recursivamente em partes menores e, posteriormente, essas partes são intercaladas em ordem crescente. Como todos os elementos precisam ser processados durante as etapas de intercalação, o algoritmo apresenta complexidade temporal $\Theta(n \log n)$ nos casos analisados.

Os resultados também permitem observar que o aumento do tamanho da entrada provoca um crescimento no tempo de execução. Entretanto, esse crescimento tende a ser menos acentuado do que o observado nos algoritmos com complexidade quadrática, especialmente para vetores de grandes dimensões.

Além disso, a implementação utiliza vetores auxiliares durante a intercalação dos elementos, necessitando de memória adicional proporcional ao tamanho da entrada. Dessa forma, o Merge Sort apresenta um bom desempenho para grandes conjuntos de dados, embora utilize mais memória auxiliar do que algoritmos como o Insertion Sort.

5.6 Quick Sort

Resultados dos experimentos — Quick Sort V1

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	2e-06	1e-06	1e-06
100.0	6e-06	9e-06	8e-06
1000.0	0.000418	0.000412	7.9e-05
10000.0	0.031878	0.03225	0.000946
100000.0	3.09547	3.2159	0.010301
1000000.0	369.153	382.153	0.097165

Figura 11: Tabela de tempo de execução do algoritmo Quick Sort (versão 1, pivô no primeiro elemento)

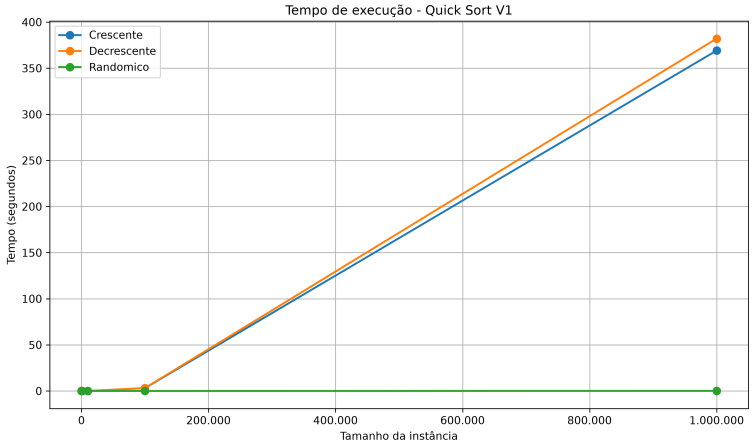


Figura 12: Gráfico de tempo de execução do algoritmo Quick Sort (versão 1, pivô no primeiro elemento)

Resultados dos experimentos — Quick Sort V2

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	1e-06	1e-06	1e-06
100.0	3e-06	3e-06	9e-06
1000.0	2.6e-05	2.7e-05	0.000104
10000.0	0.000369	0.000363	0.001285
100000.0	0.004678	0.004522	0.016285
1000000.0	0.050352	0.053225	0.144711

Figura 13: Tabela de tempo de execução do algoritmo Quick Sort (versão 2, pivô pela média aritmética)

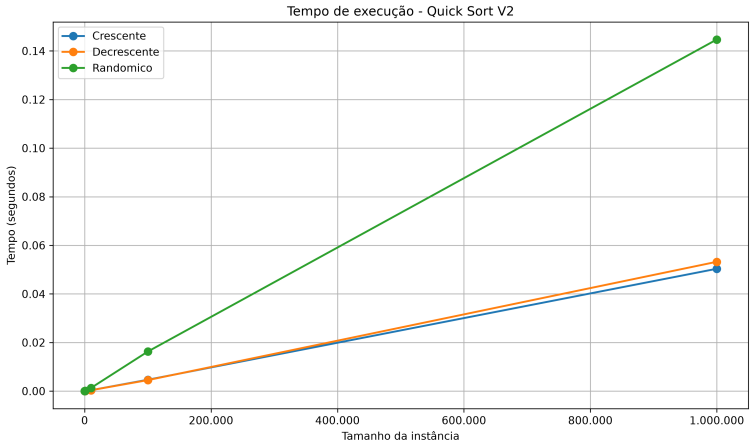


Figura 14: Gráfico de tempo de execução do algoritmo Quick Sort (versão 2, pivô pela média aritmética)

Os resultados obtidos nos experimentos estão apresentados nas tabelas e nos gráficos de cada versão. Diferentemente do Merge Sort, o desempenho do Quick Sort depende da organização inicial dos elementos, pois ela influencia a escolha do pivô e, consequentemente, o balanceamento das partições geradas a cada particionamento.

Na versão 1, o pivô é sempre o primeiro elemento da partição. Para entradas aleatórias, é esperado que as partições fiquem razoavelmente equilibradas, resultando em um comportamento próximo de $\Theta(n \log n)$. Já para entradas crescentes e decrescentes, o pivô tende a ser o menor ou o maior elemento da partição, gerando partições muito desiguais e aproximando o algoritmo do pior caso, com complexidade $\Theta(n^2)$. Esse efeito pode ser observado na Figura 12, em que o tempo de execução dessas entradas cresce de forma mais acentuada com o aumento do tamanho do vetor.

Na versão 2, o pivô é a média aritmética dos elementos da partição. Como a média tende a ficar próxima do centro dos valores quando eles estão distribuídos de forma uniforme, as entradas crescentes e decrescentes deixam de gerar partições tão desbalanceadas, e o tempo de execução dessas entradas se aproxima do observado para entradas aleatórias, como mostra a Figura 14. Em contrapartida, o cálculo da média exige percorrer toda a partição a cada chamada, o que acrescenta um custo linear adicional ao particionamento.

Comparando as duas versões, observa-se que a versão 2 reduz a sensibilidade do algoritmo à ordem inicial do vetor, enquanto a versão 1 é mais simples e não possui custo adicional para a escolha do pivô, mas é mais prejudicada por entradas já ordenadas. Para entradas aleatórias, as duas versões apresentam comportamento semelhante, com a versão 2 ligeiramente mais custosa devido ao cálculo da média.

Além disso, o algoritmo realiza a ordenação no próprio vetor, sem necessidade de vetores auxiliares. A implementação realiza a chamada recursiva apenas sobre a menor partição, o que limita a profundidade da pilha de recursão a, no máximo, proporcional a $\log_2 n$. Dessa forma, o Quick Sort utiliza menos memória auxiliar do que o Merge Sort, embora possa apresentar desempenho inferior em seu pior caso.

6 COMPARAÇÃO ENTRE OS ALGORITMOS

Além da análise individual de cada algoritmo, foram realizados experimentos comparativos entre o Insertion Sort, Bubble Sort, Selection Sort, Shell Sort, Merge Sort e Quick Sort (em 2 versões diferentes) considerando os três tipos de entrada utilizados nos experimentos: crescente, decrescente e aleatória.

6.1 Entrada crescente

A Figura 15 apresenta a comparação entre os sete algoritmos para entradas inicialmente ordenadas de forma crescente.

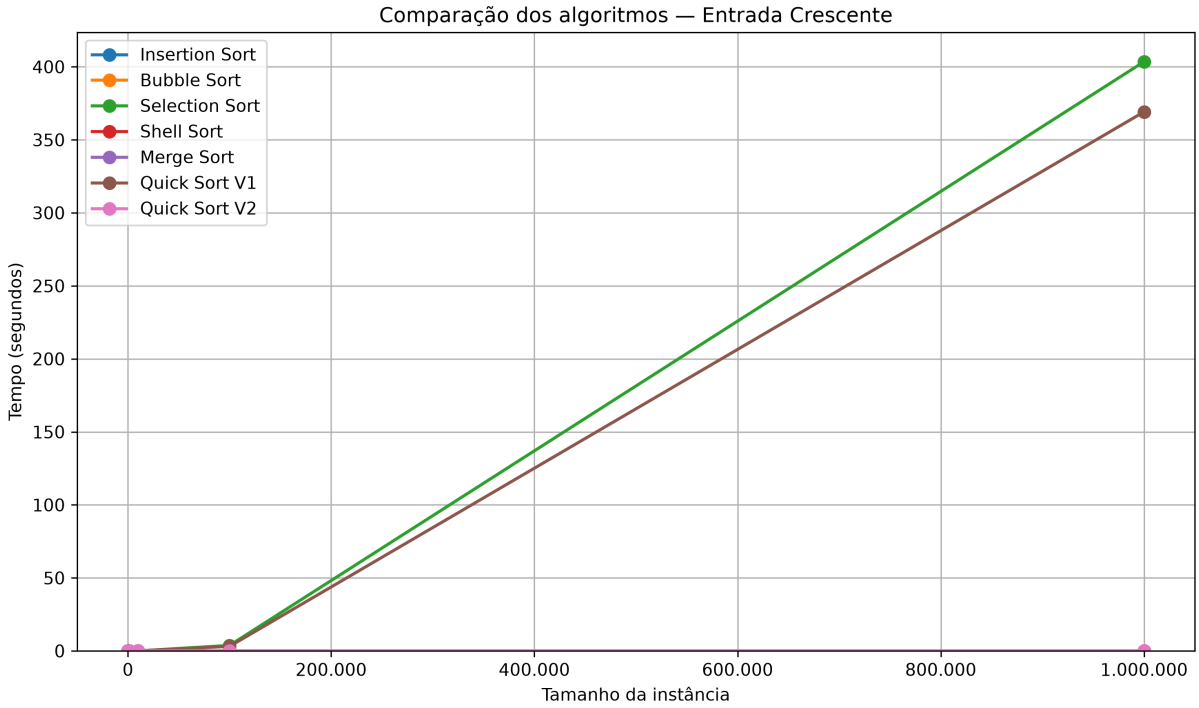


Figura 15: Comparação dos algoritmos para entrada crescente.

Nesse tipo de entrada, apenas dois algoritmos apresentaram tempos elevados: o *Selection Sort*, com cerca de 400 segundos para 1.000.000 de elementos, e o *Quick Sort* versão 1, com cerca de 370 segundos. O *Selection Sort* mantém uma quantidade quadrática de comparações mesmo com o vetor já ordenado, pois sempre percorre a parte não ordenada para localizar o menor elemento. Já o *Quick Sort* versão 1 utiliza o primeiro elemento como pivô, que em um vetor crescente é sempre o menor valor da partição. Dessa forma, cada particionamento separa apenas um elemento, e o algoritmo se comporta de acordo com o pior caso, $\Theta(n^2)$.

O *Insertion Sort* e o *Bubble Sort* apresentaram tempos praticamente nulos na escala do gráfico, pois, em vetores já ordenados, o primeiro não precisa deslocar elementos e o segundo é interrompido antecipadamente quando não ocorrem trocas, ambos com custo linear. O *Shell Sort* e o *Merge Sort* também apresentaram tempos muito baixos, compatíveis com a complexidade próxima de $\Theta(n \log n)$.

A versão 2 do *Quick Sort* não sofreu a degradação observada na versão 1. Como a média aritmética de valores distribuídos uniformemente fica próxima do centro da partição, as partições permaneceram equilibradas mesmo com o vetor ordenado, e o tempo de execução foi comparável ao do *Merge Sort*. Esse resultado mostra que a escolha do pivô é decisiva para o desempenho do algoritmo.

6.2 Entrada decrescente

A Figura 16 apresenta a comparação entre os sete algoritmos para entradas inicialmente organizadas em ordem decrescente.

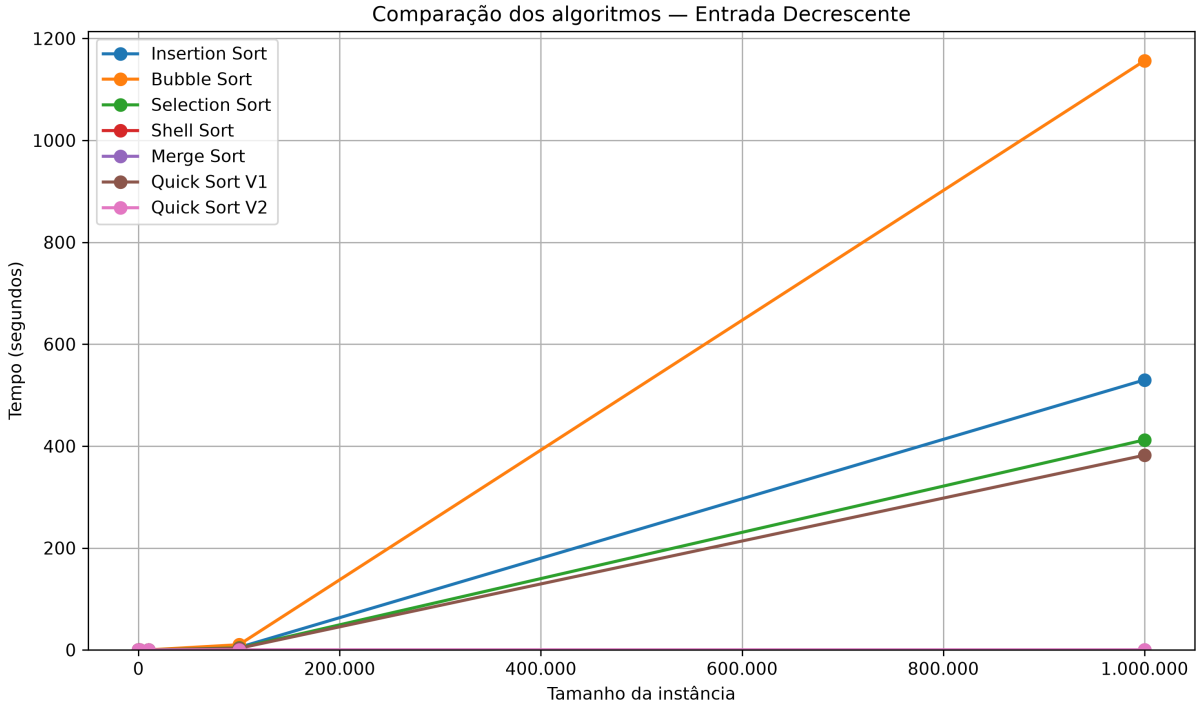


Figura 16: Comparação dos algoritmos para entrada decrescente.

Nesse caso, quatro algoritmos apresentaram tempos elevados para 1.000.000 de elementos. O *Bubble Sort* foi o mais lento, com cerca de 1150 segundos, devido à grande quantidade de comparações e trocas necessárias para inverter completamente o vetor. Em

seguida aparece o *Insertion Sort*, com cerca de 530 segundos, pois cada novo elemento precisa ser deslocado até o início da parte já ordenada. Esse valor é aproximadamente o dobro do observado em entradas aleatórias, o que é coerente com o fato de o número de deslocamentos no pior caso ser cerca de duas vezes o do caso médio.

O *Selection Sort*, com cerca de 410 segundos, apresentou tempo muito próximo ao observado nas entradas crescente e aleatória, pois o número de comparações realizadas não depende da organização inicial do vetor. O *Quick Sort* versão 1 apresentou cerca de 380 segundos: em um vetor decrescente, o primeiro elemento é o maior valor da partição, e o particionamento separa apenas um elemento por vez, o que leva o algoritmo ao pior caso quadrático.

O *Shell Sort*, o *Merge Sort* e o *Quick Sort* versão 2 apresentaram tempos muito baixos, praticamente indistinguíveis do eixo horizontal na escala do gráfico. No caso do *Merge Sort*, isso ocorre porque as etapas de divisão e intercalação são as mesmas independentemente da disposição inicial dos elementos. Na versão 2 do *Quick Sort*, a média aritmética manteve as partições equilibradas, evitando o pior caso observado na versão 1.

6.3 Entrada aleatória

A Figura 17 apresenta a comparação entre os sete algoritmos implementados (os seis algoritmos estudados, sendo o *Quick Sort* em suas duas versões) para entradas geradas aleatoriamente.

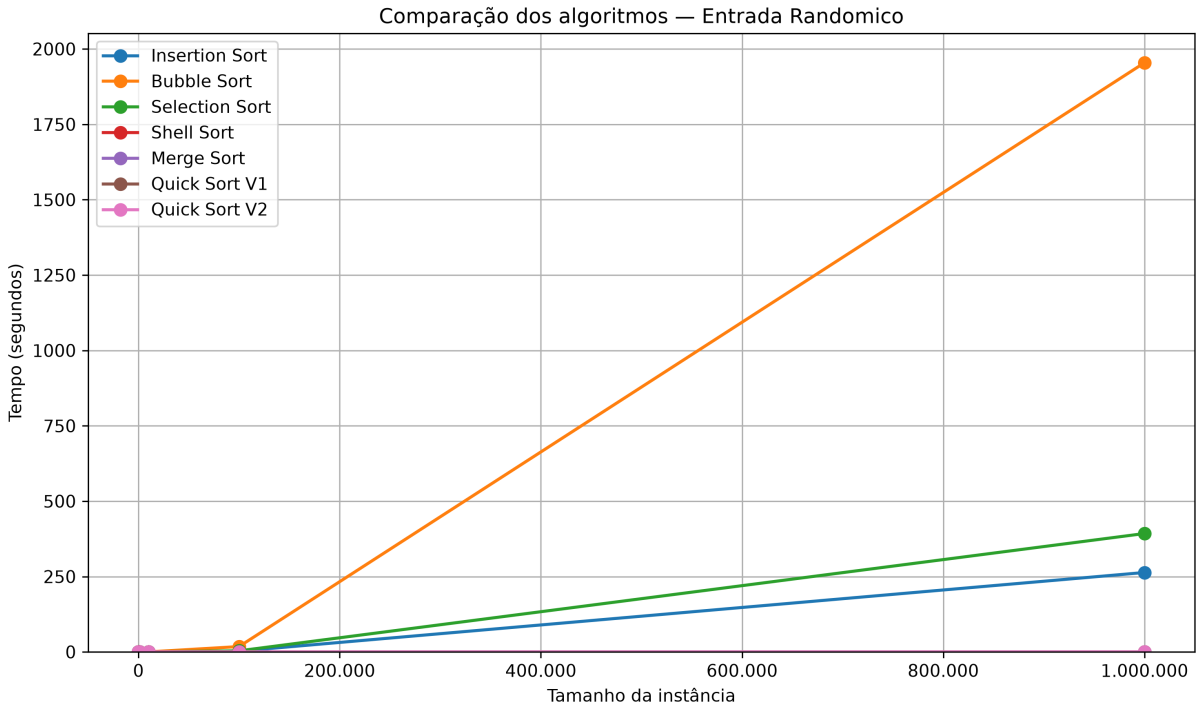


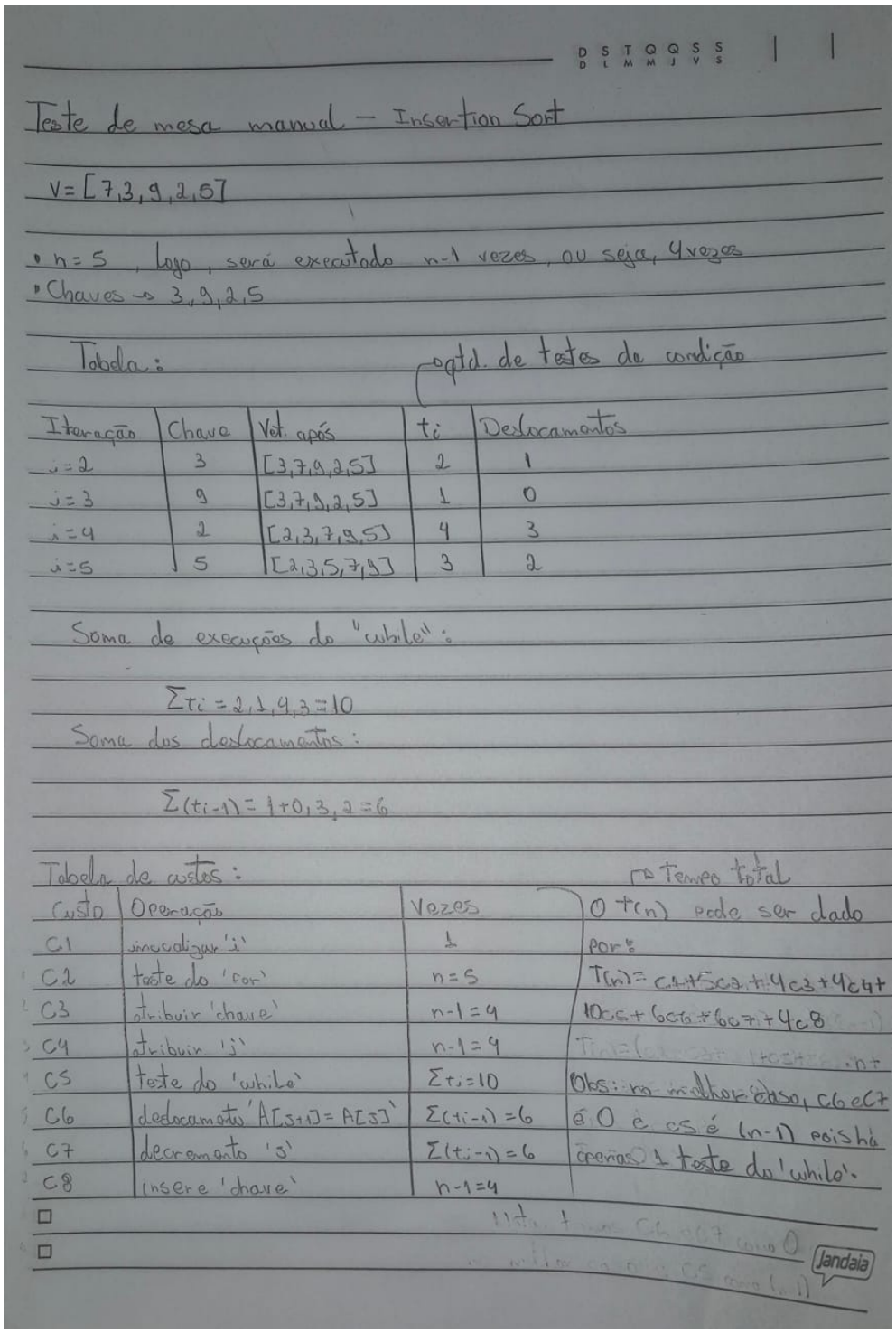
Figura 17: Comparação dos algoritmos para entrada aleatória.

Para as entradas aleatórias, observa-se uma separação clara entre os algoritmos de custo quadrático e os demais. Para instâncias de 1.000.000 de elementos, o *Bubble Sort* apresenta o maior tempo de execução, em torno de 1950 segundos, seguido pelo *Selection Sort*, com cerca de 390 segundos, e pelo *Insertion Sort*, com cerca de 260 segundos. Esses resultados são coerentes com o crescimento quadrático esperado no caso médio. A menor quantidade de deslocamentos do *Insertion Sort* em relação às trocas do *Bubble Sort* explica a diferença entre eles, mesmo com a mesma ordem de complexidade.

O *Shell Sort*, o *Merge Sort* e as duas versões do *Quick Sort* apresentaram tempos muito inferiores aos dos algoritmos quadráticos, praticamente indistinguíveis do eixo horizontal na escala do gráfico. Isso confirma, na prática, a vantagem dos algoritmos com custo próximo de $\Theta(n \log n)$ para instâncias grandes. No caso do *Quick Sort*, as duas versões apresentaram comportamento semelhante em entradas aleatórias, pois, nessa situação, o primeiro elemento e a média aritmética tendem a gerar partições razoavelmente equilibradas.

7 ATIVIDADES MANUSCRITAS

7.1 Insertion Sort



Teste de mesa manual – Insertion Sort

V = [7,3,9,2,5]

- n = 5, logo, será executado n-1 vezes, ou seja, 4 vezes
- Chaves → 3,9,2,5

Tabela: (qtd. de testes da condição)

Iteração	Chave	Vet. após	t_i	Deslocamentos
i = 2	3	[3,7,9,2,5]	2	1
i = 3	9	[3,7,9,2,5]	1	0
i = 4	2	[2,3,7,9,5]	4	3
i = 5	5	[2,3,5,7,9]	3	2

Soma de execuções do "while":

$\Sigma t_i = 2,1,4,3 = 10$

Soma dos deslocamentos:

$\Sigma (t_i - 1) = 1+0,3,2 = 6$

Tabela de custos: (o tempo total)

Custo	Operação	Vezes	O T(n) pode ser dado
C1	inicializar 'i'	1	por:
C2	teste do 'for'	n = 5	T(n) = C1+5C2+4C3+4C4+
C3	atribuir 'chave'	n-1 = 4	10C5+6C6+6C7+4C8
C4	atribuir 'j'	n-1 = 4	
C5	teste do 'while'	$\Sigma t_i = 10$	Obs: no melhor caso, C6 e C7
C6	deslocamento 'A[j+1] = A[j]'	$\Sigma (t_i - 1) = 6$	é 0 e C5 é (n-1) pois há
C7	decremento 'j'	$\Sigma (t_i - 1) = 6$	apenas 1 teste do 'while'.
C8	insere 'chave'	n-1 = 4	

Figura 18: Teste de mesa manual do algoritmo Insertion Sort

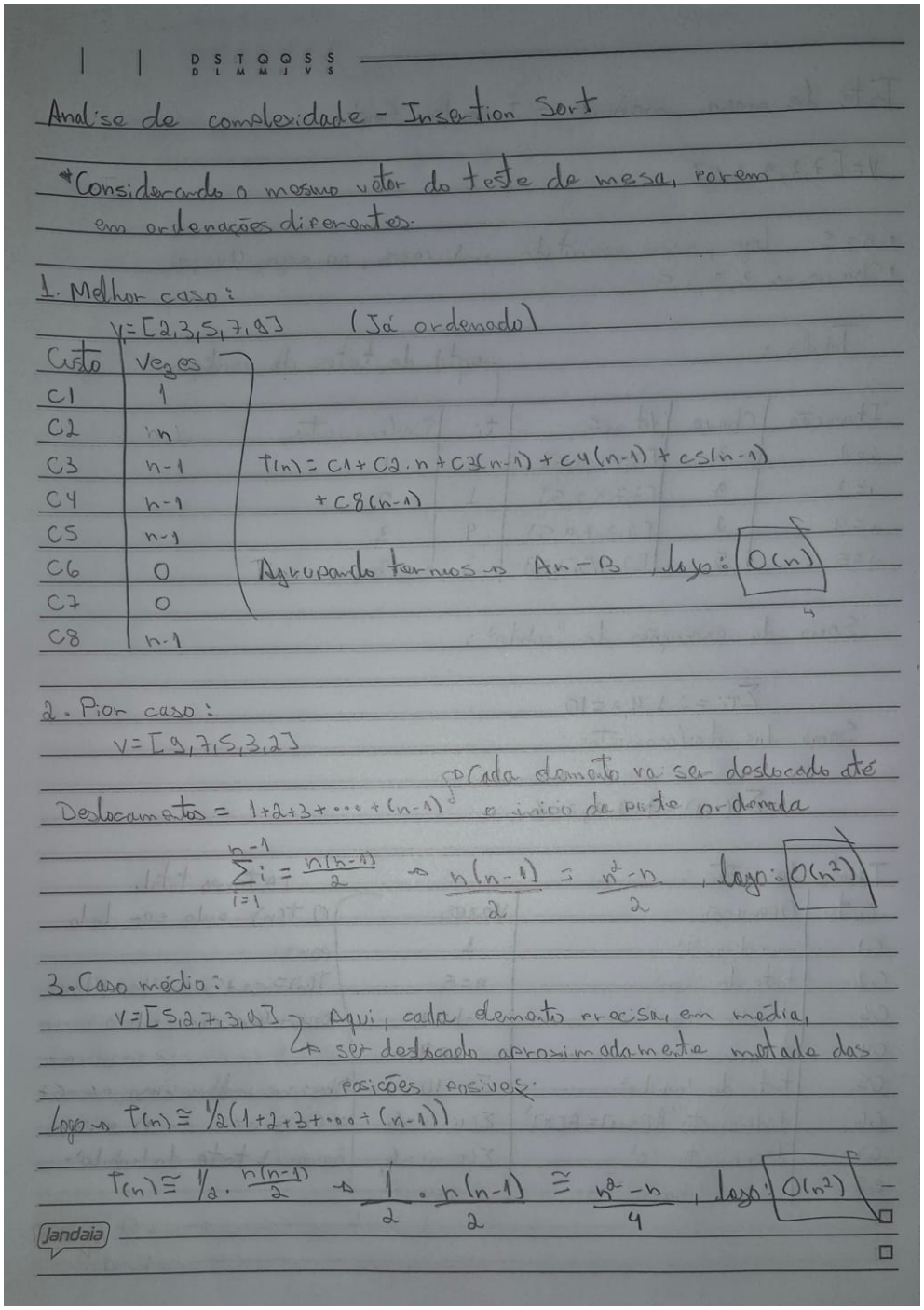
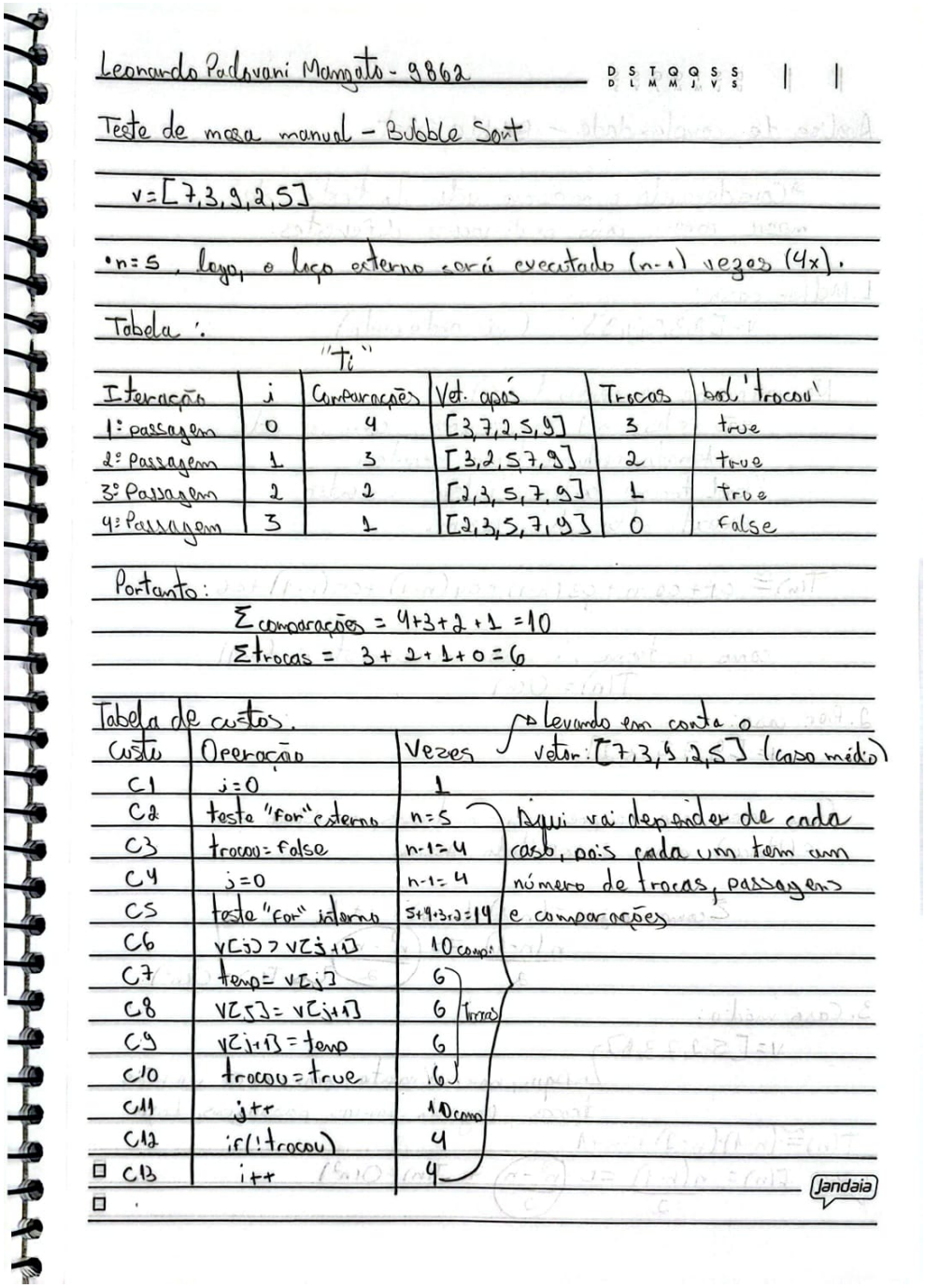


Figura 19: Análise de complexidade do algoritmo Insertion Sort

7.2 Bubble Sort



Leonardo Padovani Mangoto - 9862

Teste de mesa manual - Bubble Sort

v=[7,3,9,2,5]

• n=5, logo, o laço externo será executado (n-1) vezes (4x).

Tabela 1:

Iteração	i	Comparações "Ti"	Vet. após	Trocas	bool 'trocou'
1ª passagem	0	4	[3,7,2,5,9]	3	true
2ª passagem	1	3	[3,2,5,7,9]	2	true
3ª Passagem	2	2	[2,3,5,7,9]	1	true
4ª Passagem	3	1	[2,3,5,7,9]	0	false

Portanto:

Σ comparações = 4+3+2+1 = 10

Σ trocas = 3+2+1+0 = 6

Tabela de custos. (levando em conta o vetor: [7,3,9,2,5] (caso médio))

Custo	Operação	Vezes	
C1	i=0	1	
C2	teste "for" externo	n=5	Aqui vai depender de cada caso, pois cada um tem um número de trocas, passagens e comparações
C3	trocou=false	n-1=4	
C4	j=0	n-1=4	
C5	teste "for" interno	5+4+3+2=14	
C6	v[j] > v[j+1]	10 comp.	
C7	temp=v[j]	6	Trocas
C8	v[j]=v[j+1]	6	
C9	v[j+1]=temp	6	
C10	trocou=true	6	
C11	j++	10 comp.	
C12	if(!trocou)	4	
C13	i++	4	

Figura 20: Teste de mesa manual do algoritmo Bubble Sort

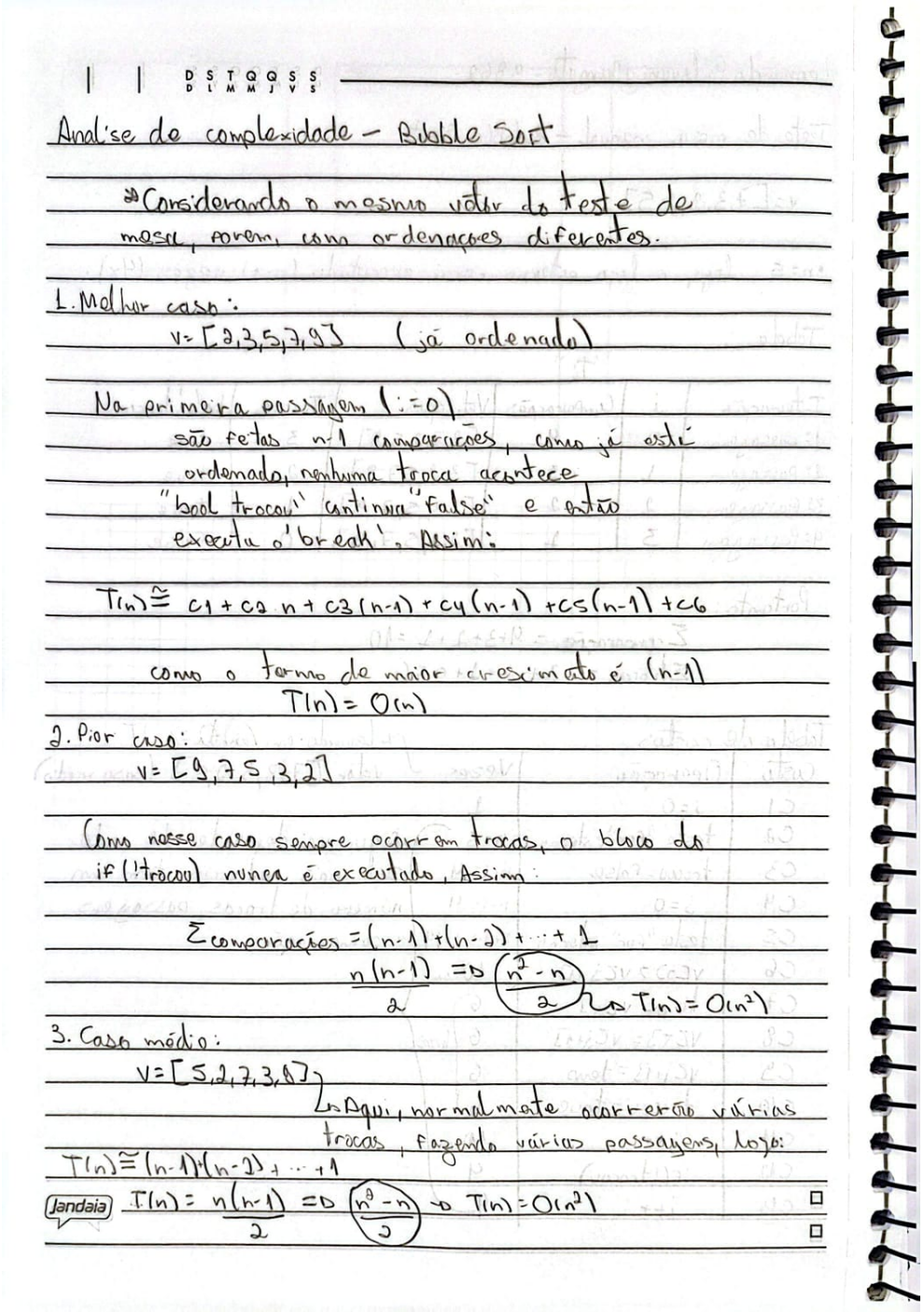
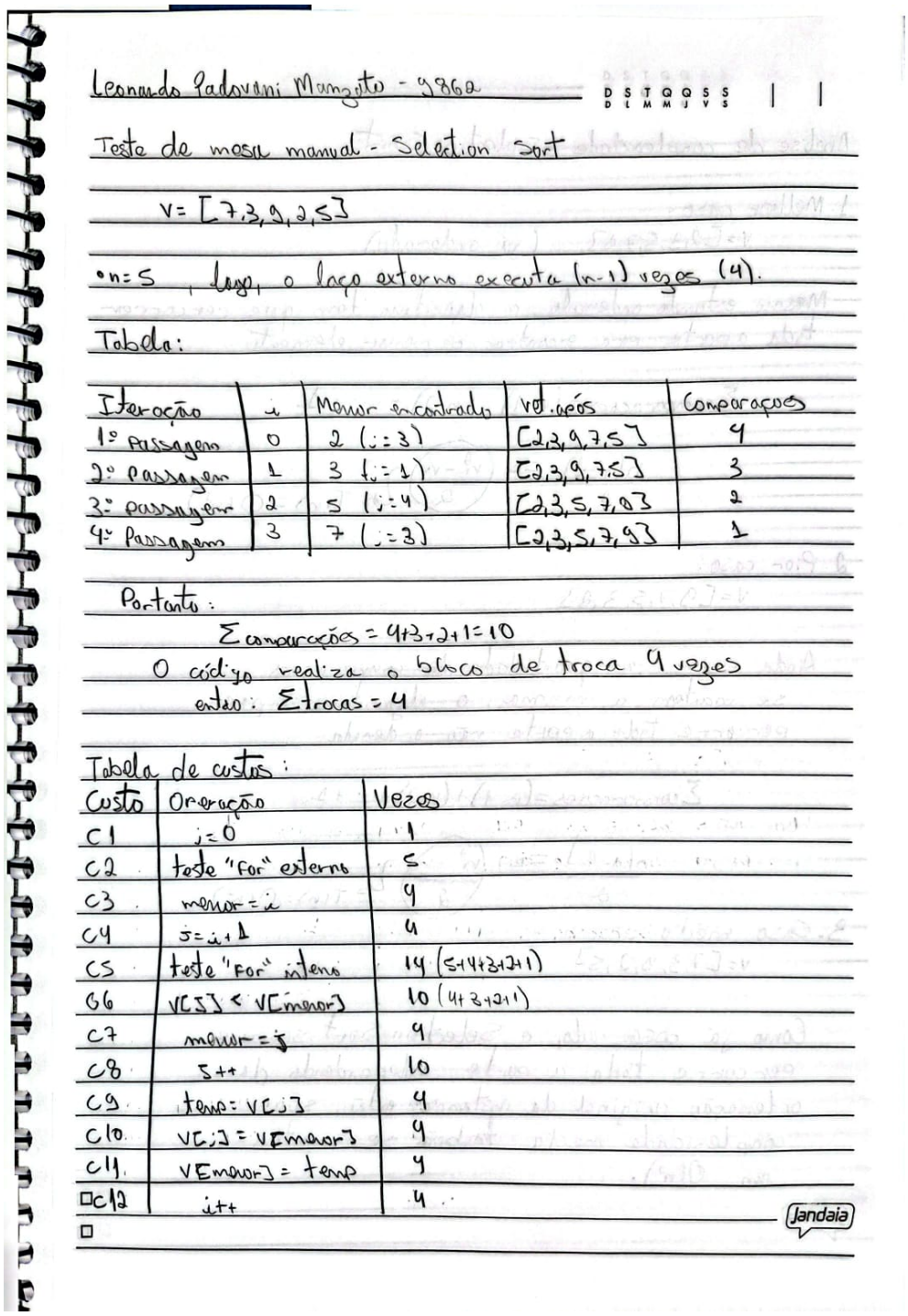


Figura 21: Análise de complexidade do algoritmo Bubble Sort

7.3 Selection Sort



Leonardo Padovani Mangoto - 9868

Teste de mesa manual - Selection Sort

V = [7,3,9,2,5]

• n = 5, logo, o laço externo executa (n-1) vezes (4).

Tabela:

Iteração	i	Menor encontrado	vet. após	Comparações
1ª Passagem	0	2 (j=3)	[2,3,9,7,5]	4
2ª Passagem	1	3 (j=1)	[2,3,9,7,5]	3
3ª passagem	2	5 (j=4)	[2,3,5,7,9]	2
4ª Passagem	3	7 (j=3)	[2,3,5,7,9]	1

Portanto:

Σ comparações = 4+3+2+1 = 10

O código realiza o bloco de troca 4 vezes

então: Σ trocas = 4

Tabela de custos:

Custo	Operação	Vezes
C1	i = 0	1
C2	teste "for" externo	5
C3	menor = i	4
C4	j = i + 1	4
C5	teste "for" interno	14 (5+4+3+2+1)
C6	V[j] < V[menor]	10 (4+3+2+1)
C7	menor = j	4
C8	j++	10
C9	temp = V[i]	4
C10	V[i] = V[menor]	4
C11	V[menor] = temp	4
C12	i++	4

Jandaia

Figura 22: Teste de mesa manual do algoritmo Selection Sort

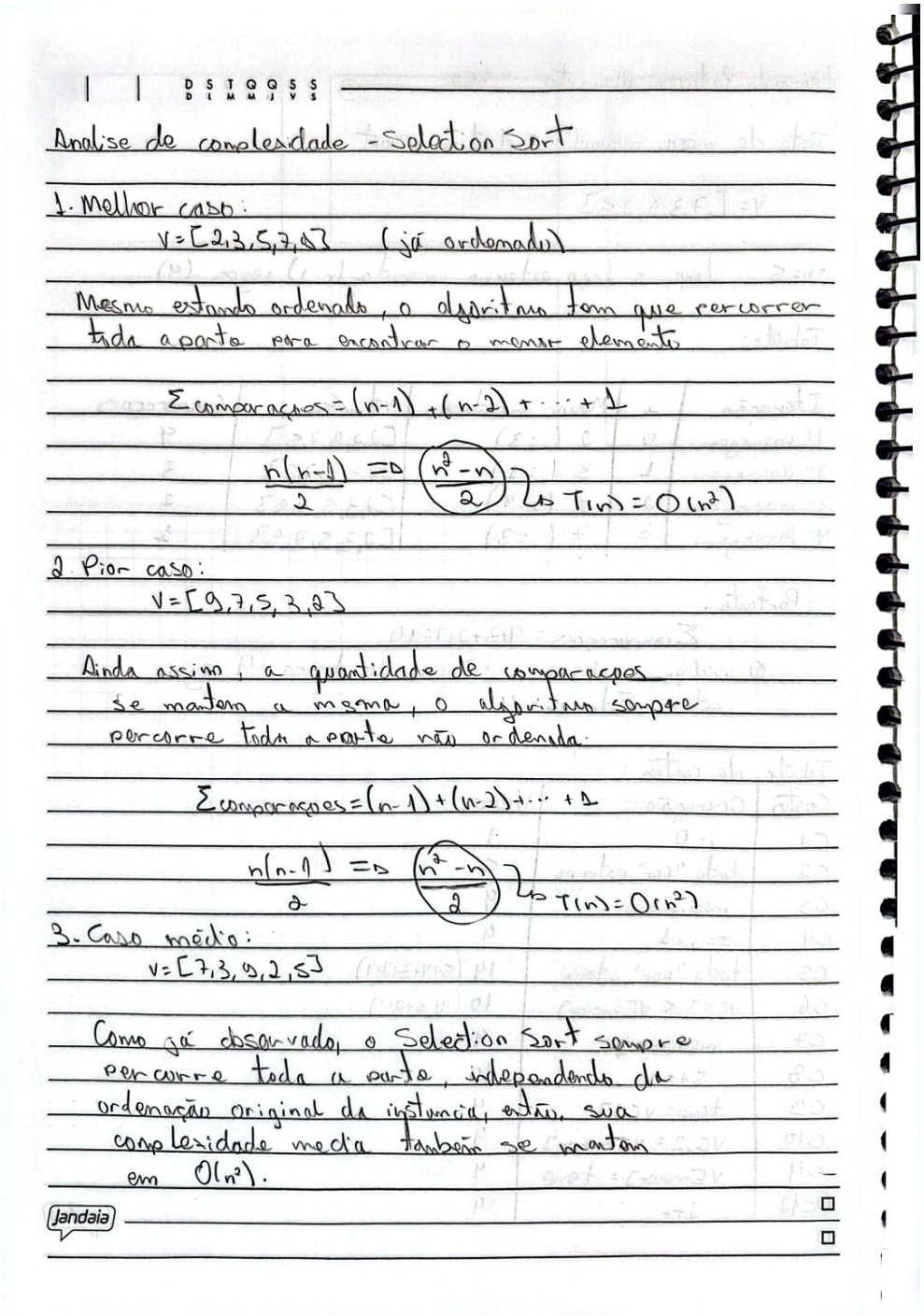
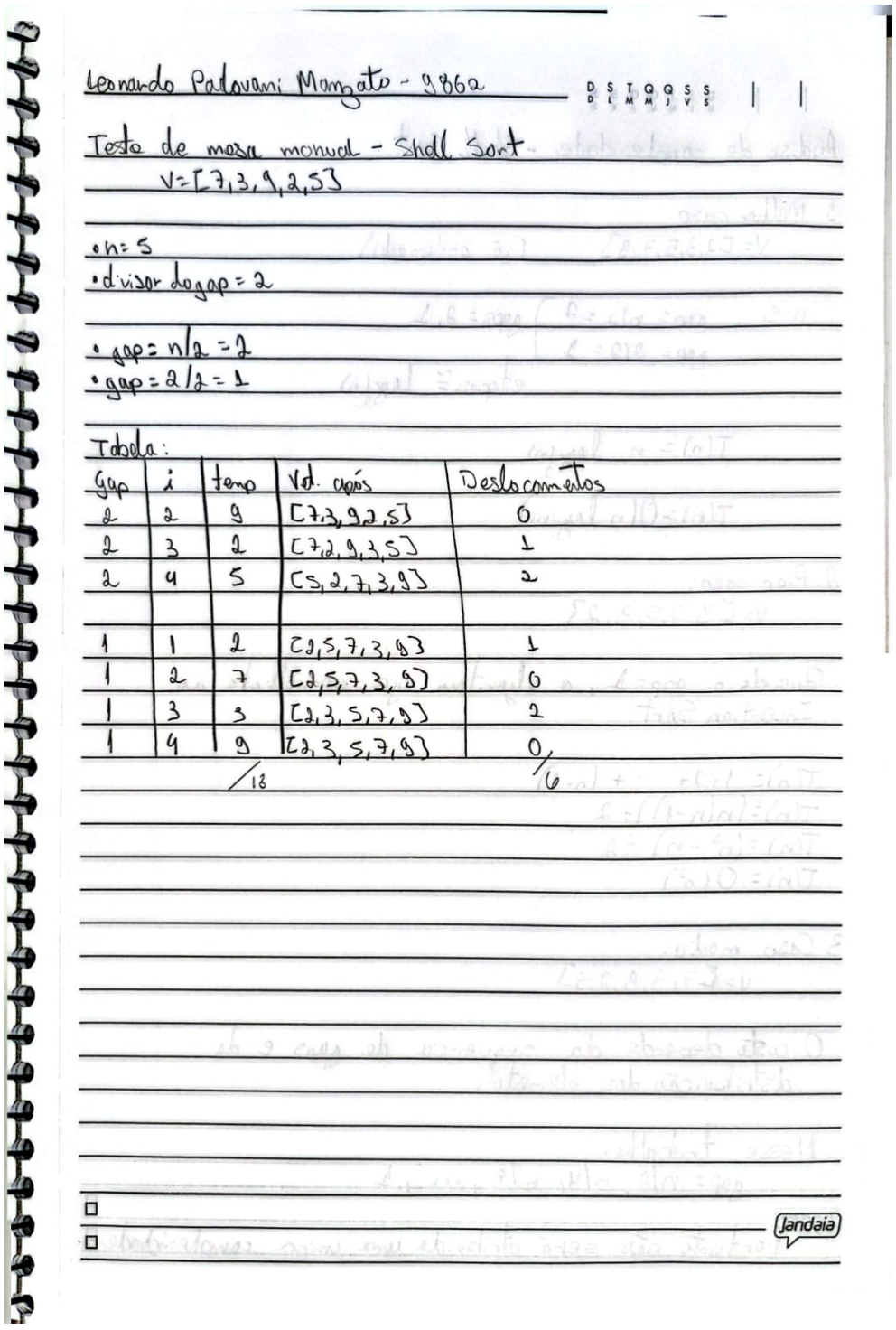


Figura 23: Análise de complexidade do algoritmo Selection Sort

7.4 Shell Sort



Leonardo Padovani Mangato - 9862

Teste de mesa manual - Shell Sort

V=[7,3,9,2,5]

- n=5
- divisor do gap = 2
- gap = n/2 = 2
- gap = 2/2 = 1

Tabela:

Gap	i	temp	Vet. após	Deslocamentos
2	2	9	[7,3,9,2,5]	0
2	3	2	[7,2,9,3,5]	1
2	4	5	[5,2,7,3,9]	2
1	1	2	[2,5,7,3,9]	1
1	2	7	[2,5,7,3,9]	0
1	3	3	[2,3,5,7,9]	2
1	4	9	[2,3,5,7,9]	0

/18 /6

Figura 24: Teste de mesa manual do algoritmo Shell Sort

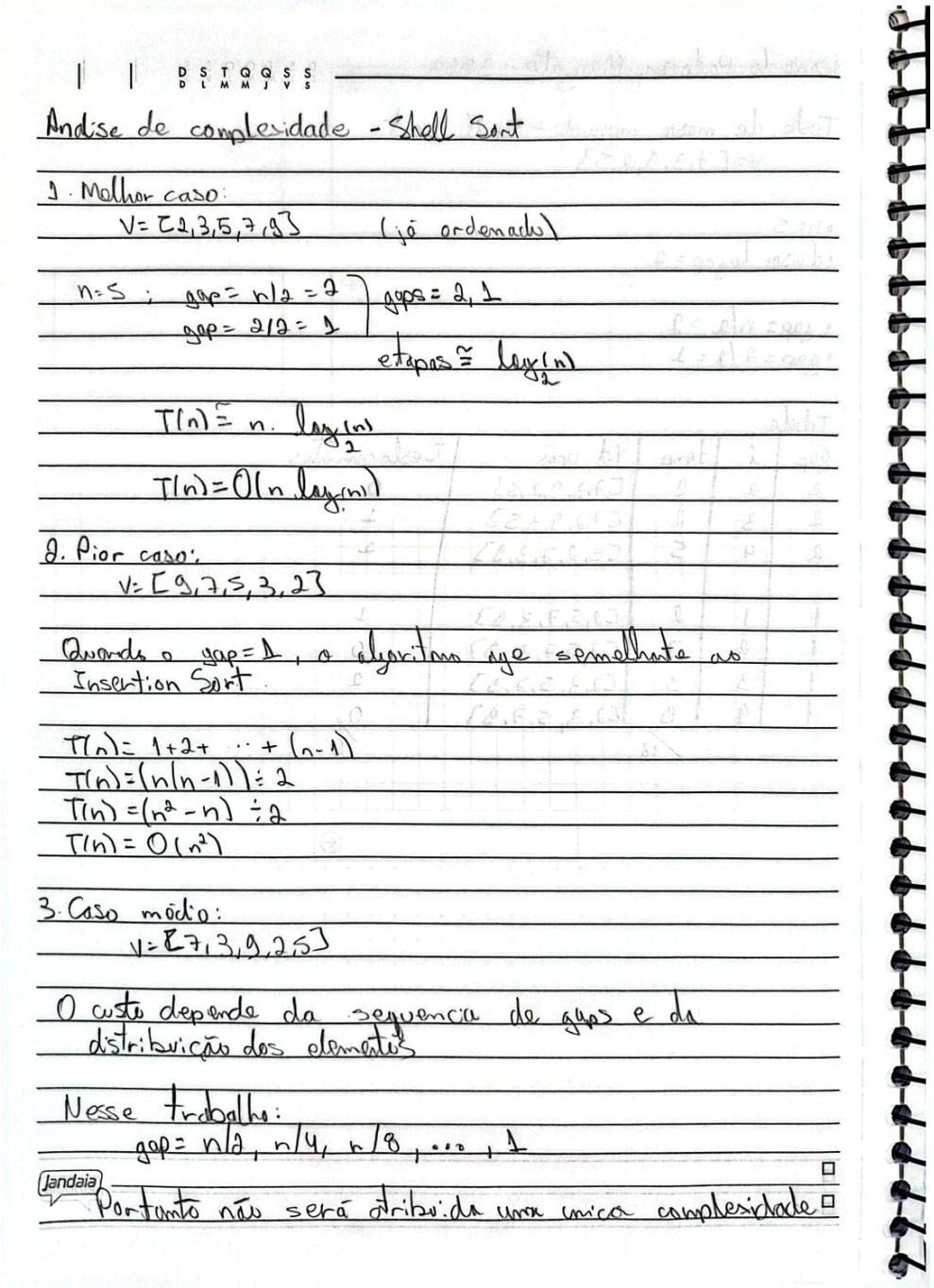


Figura 25: Análise de complexidade do algoritmo Shell Sort

7.5 Merge Sort

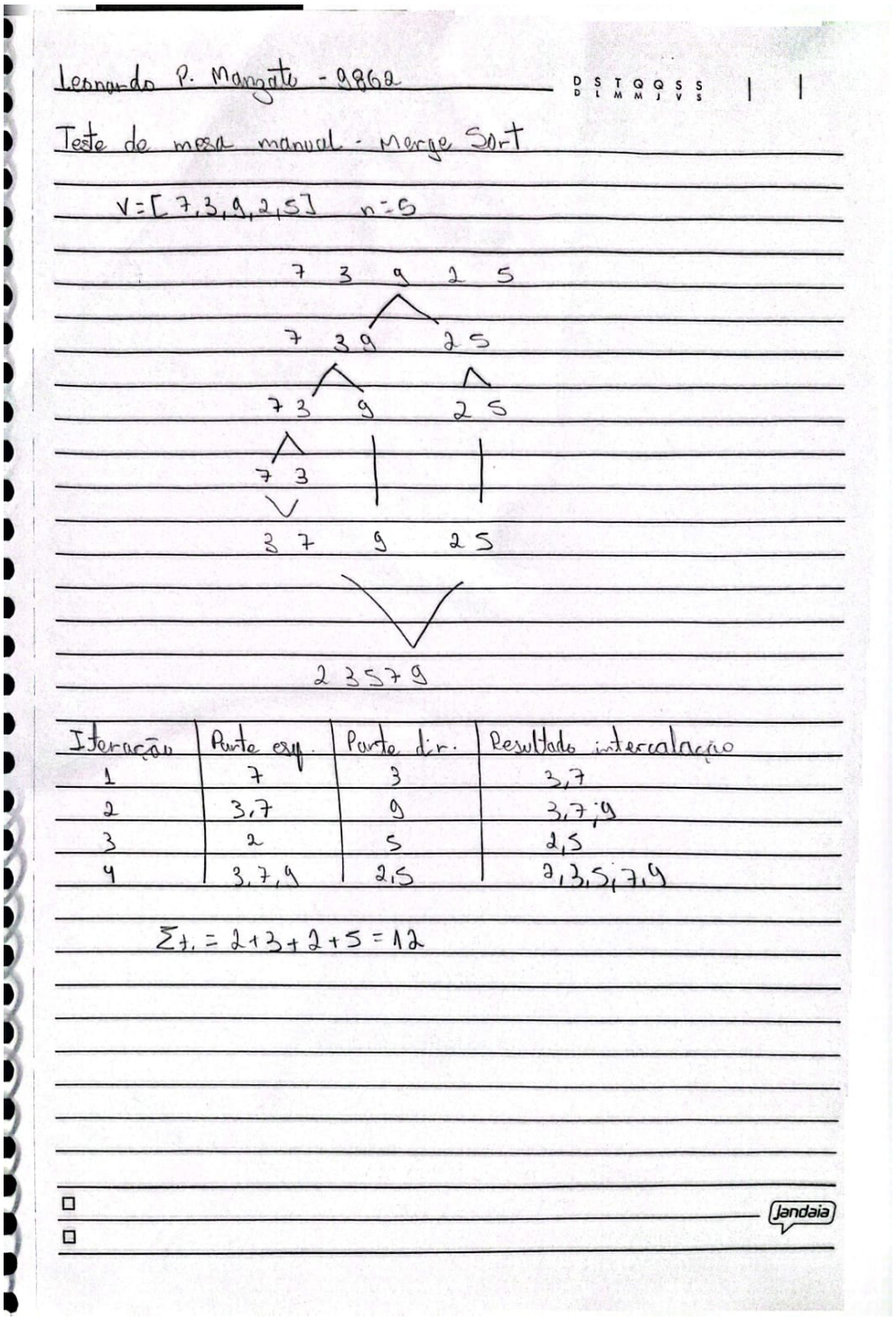


Figura 26: Teste de mesa manual do algoritmo Merge Sort

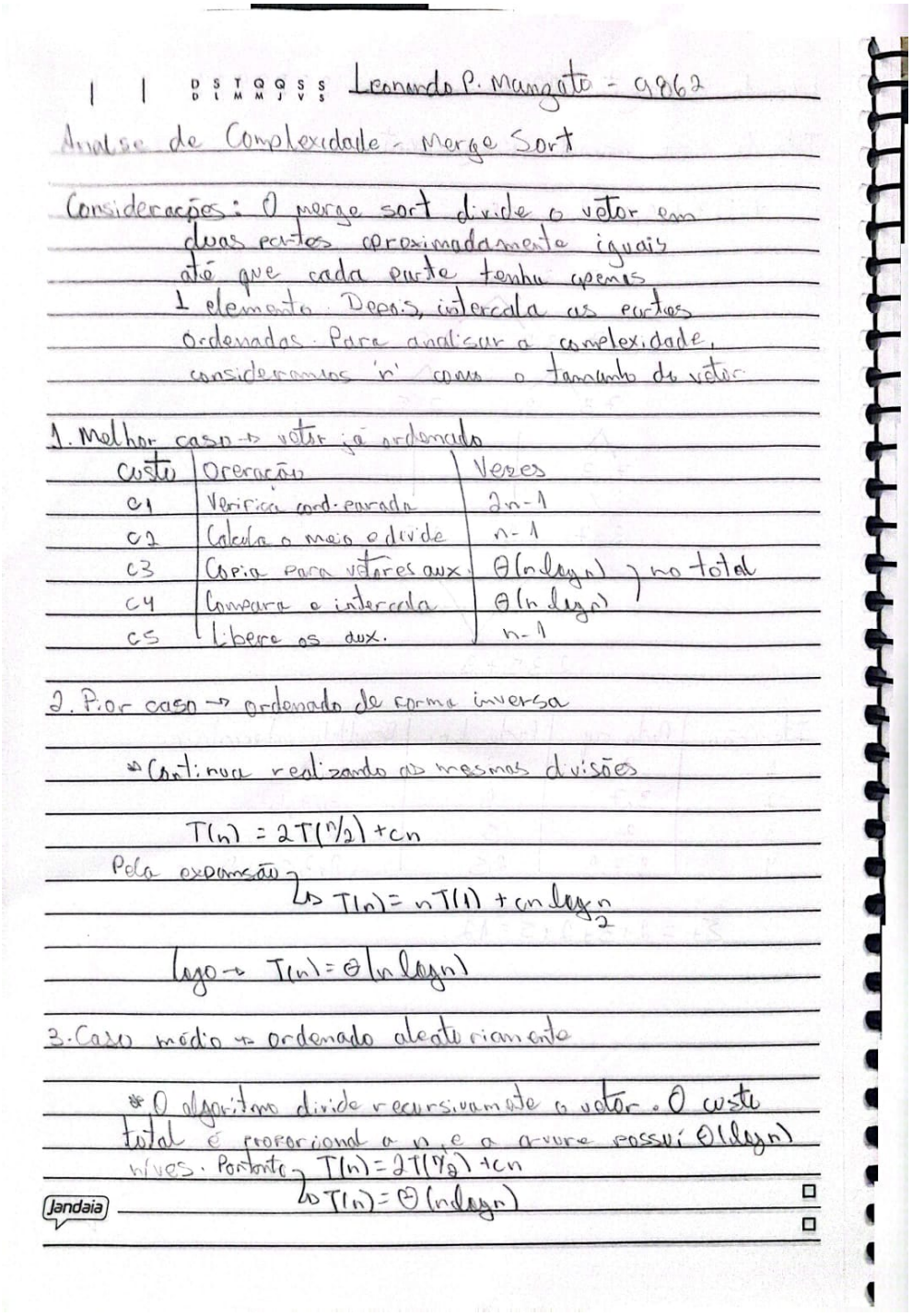
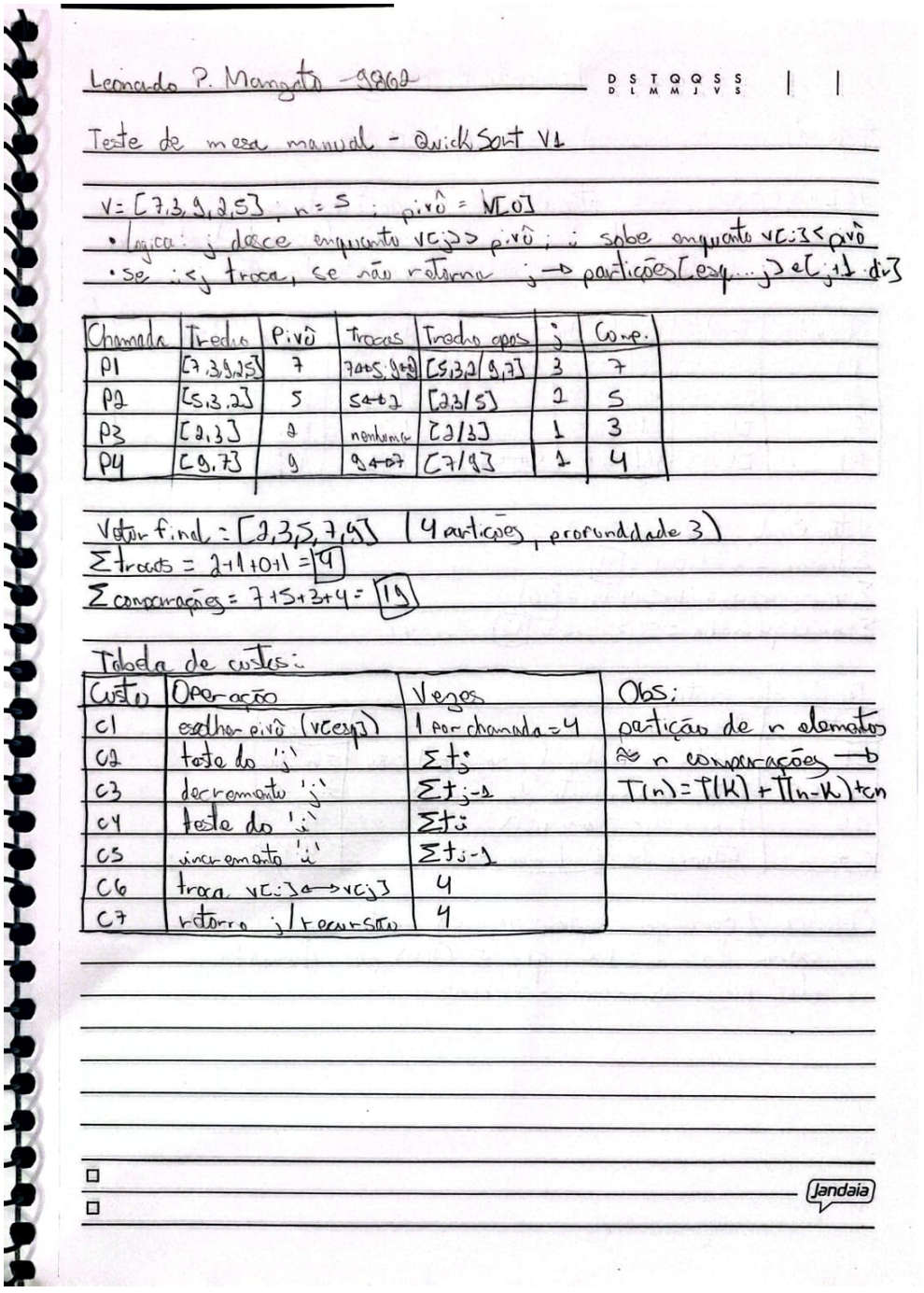


Figura 27: Análise de complexidade do algoritmo Merge Sort

7.6 Quick Sort

7.6.1 Versão 1 (pivô no primeiro elemento)



Leonardo P. Mangueira - 9960

Teste de mesa manual - QuickSort V1

V = [7,3,9,2,5] ; n = 5 ; pivô = V[0]

- lógica: j desce enquanto v[j] > pivô ; i sobe enquanto v[i] < pivô
- se i < j troca, se não retorna j → partições [esq ... j] e [j+1 ... dir]

Chamada	Trecho	Pivô	Trocas	Trecho após	j	Comp.
P1	[7,3,9,2,5]	7	7↔5; 9↔2	[5,3,2 | 9,7]	3	7
P2	[5,3,2]	5	5↔2	[2,3 | 5]	1	5
P3	[2,3]	2	nenhuma	[2 | 3]	1	3
P4	[9,7]	9	9↔7	[7 | 9]	1	4

Vetor final = [2,3,5,7,9] (4 partições, profundidade 3)

Σ trocas = 2+1+0+1 = 4

Σ comparações = 7+5+3+4 = 19

Tabela de custos:

Custo	Operação	Vezes	Obs:
c1	escolher pivô (v[esq])	1 por chamada = 4	partição de n elemntos
c2	teste do 'j'	Σ t_j	≈ n comparações →
c3	decremento 'j'	Σ $t_j - 4$	T(n) = T(k) + T(n-k) + cn
c4	teste do 'i'	Σ t_j	
c5	incremento 'i'	Σ $t_j - 1$	
c6	troca v[i] ↔ v[j]	4	
c7	retorno j / recursão	4	

Jandaia

Figura 28: Teste de mesa manual do algoritmo Quick Sort (versão 1, pivô no primeiro elemento)

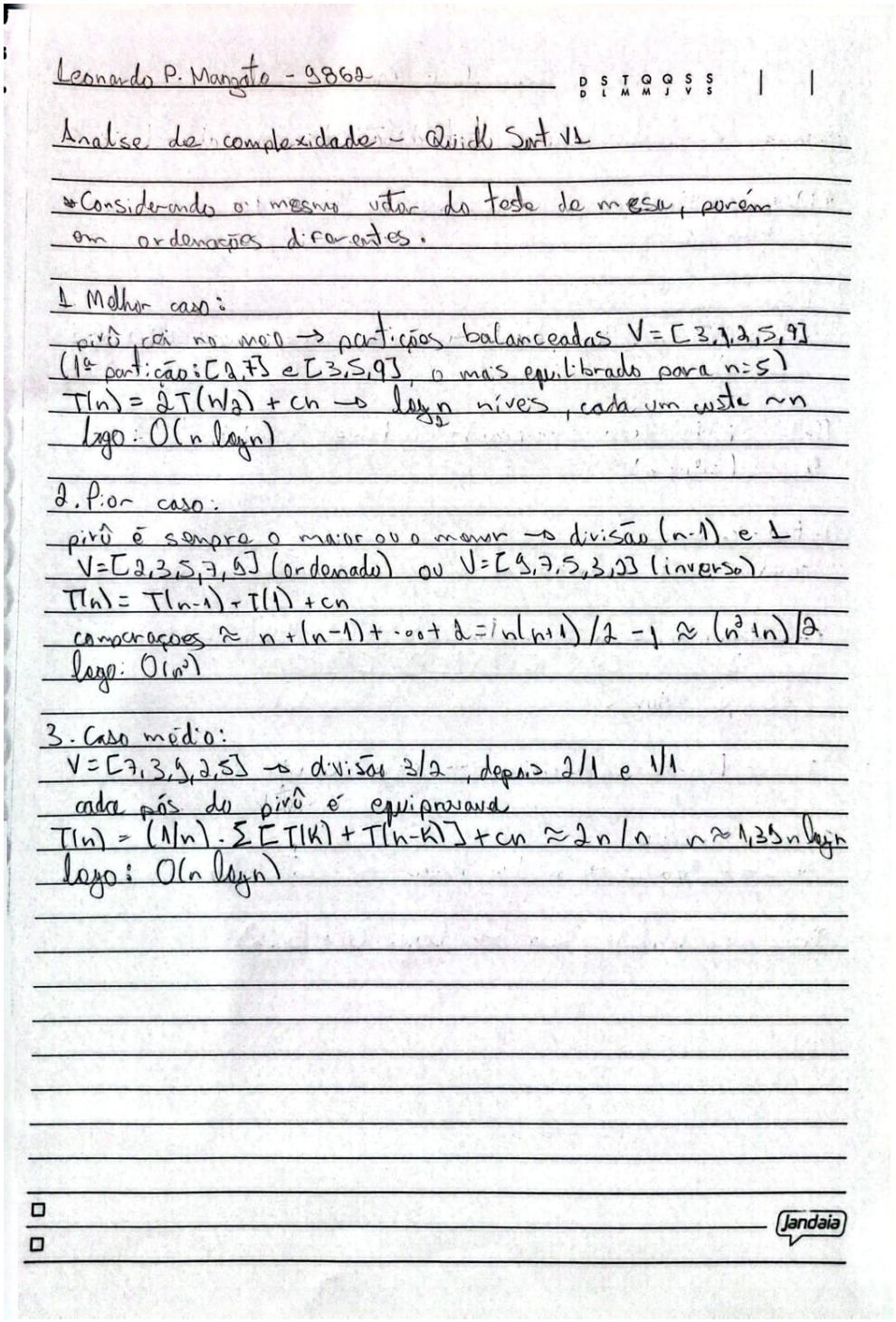
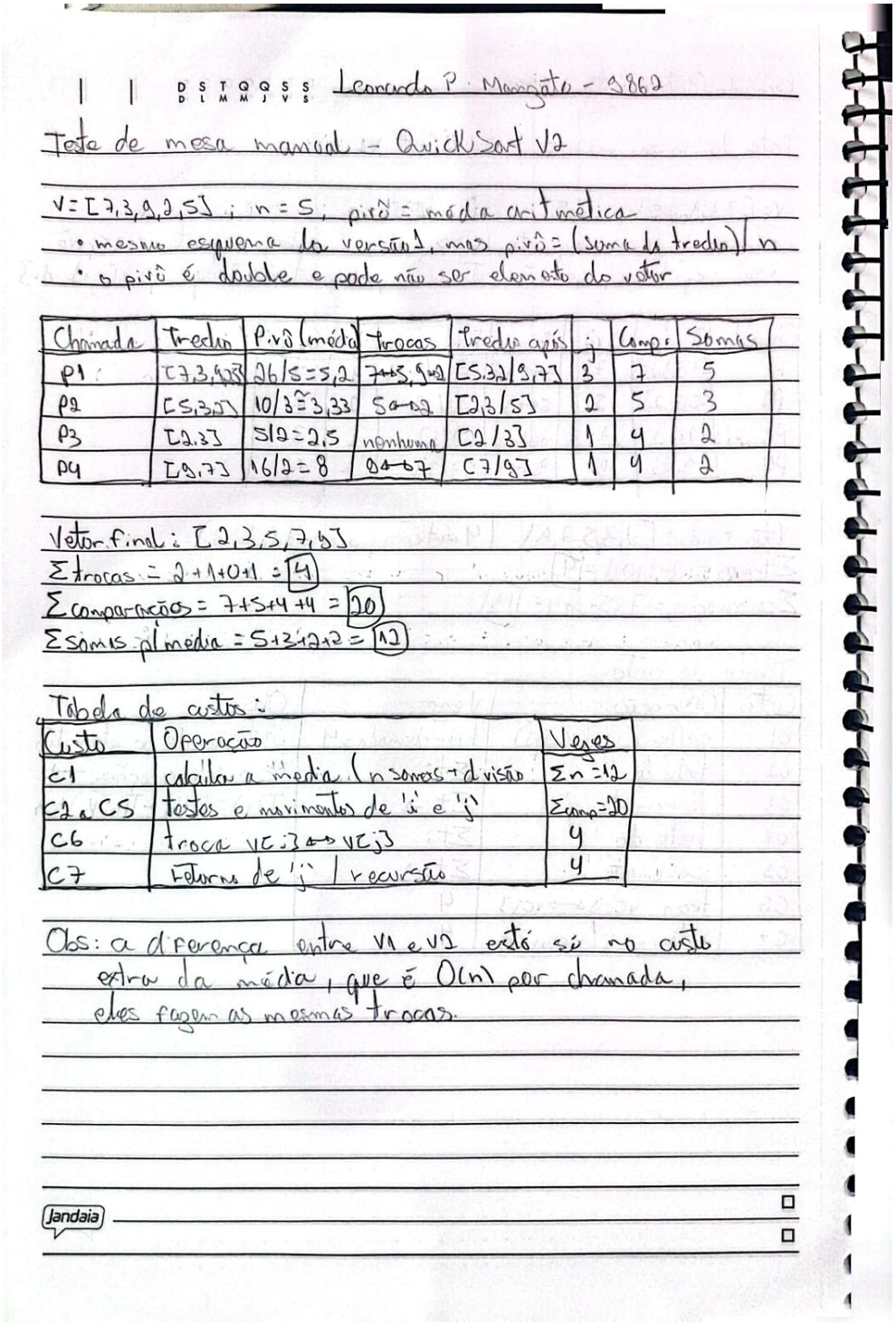


Figura 29: Análise de complexidade do algoritmo Quick Sort (versão 1, pivô no primeiro elemento)

7.6.2 Versão 2 (pivô pela média aritmética)



Leonardo P. Mangate - 9862

Teste de mesa manual - QuickSort V2

V = [7,3,9,2,5] ; n = 5 ; pivô = média aritmética

- mesmo esquema da versão 1, mas pivô = (soma do trecho) / n
- o pivô é double e pode não ser elemento do vetor

Chamada	Trecho	Pivô (média)	Trocas	Trecho após	j	Comp.	Somas
p1	[7,3,9,2,5]	26/5 = 5,2	7↔5, 9↔2	[5,3,2/9,7]	3	7	5
p2	[5,3,2]	10/3 ≈ 3,33	5↔2	[2,3/5]	2	5	3
p3	[2,3]	5/2 = 2,5	nenhuma	[2/3]	1	4	2
p4	[9,7]	16/2 = 8	9↔7	[7/9]	1	4	2

Vetor final: [2,3,5,7,9]

Σ trocas = 2 + 1 + 0 + 1 = 4

Σ comparações = 7 + 5 + 4 + 4 = 20

Σ somas p/ média = 5 + 3 + 2 + 2 = 12

Tabela de custos:

Custo	Operação	Vezes
c1	calcular a média (n somas + divisão	Σ n = 12
c2 a c5	testes e movimentos de 'i' e 'j'	Σ comp = 20
c6	troca v[i] ↔ v[j]	4
c7	retorno de 'j' / recursão	4

Obs: a diferença entre V1 e V2 está só no custo extra da média, que é O(n) por chamada, eles fazem as mesmas trocas.

Jandaia

Figura 30: Teste de mesa manual do algoritmo Quick Sort (versão 2, pivô pela média aritmética)

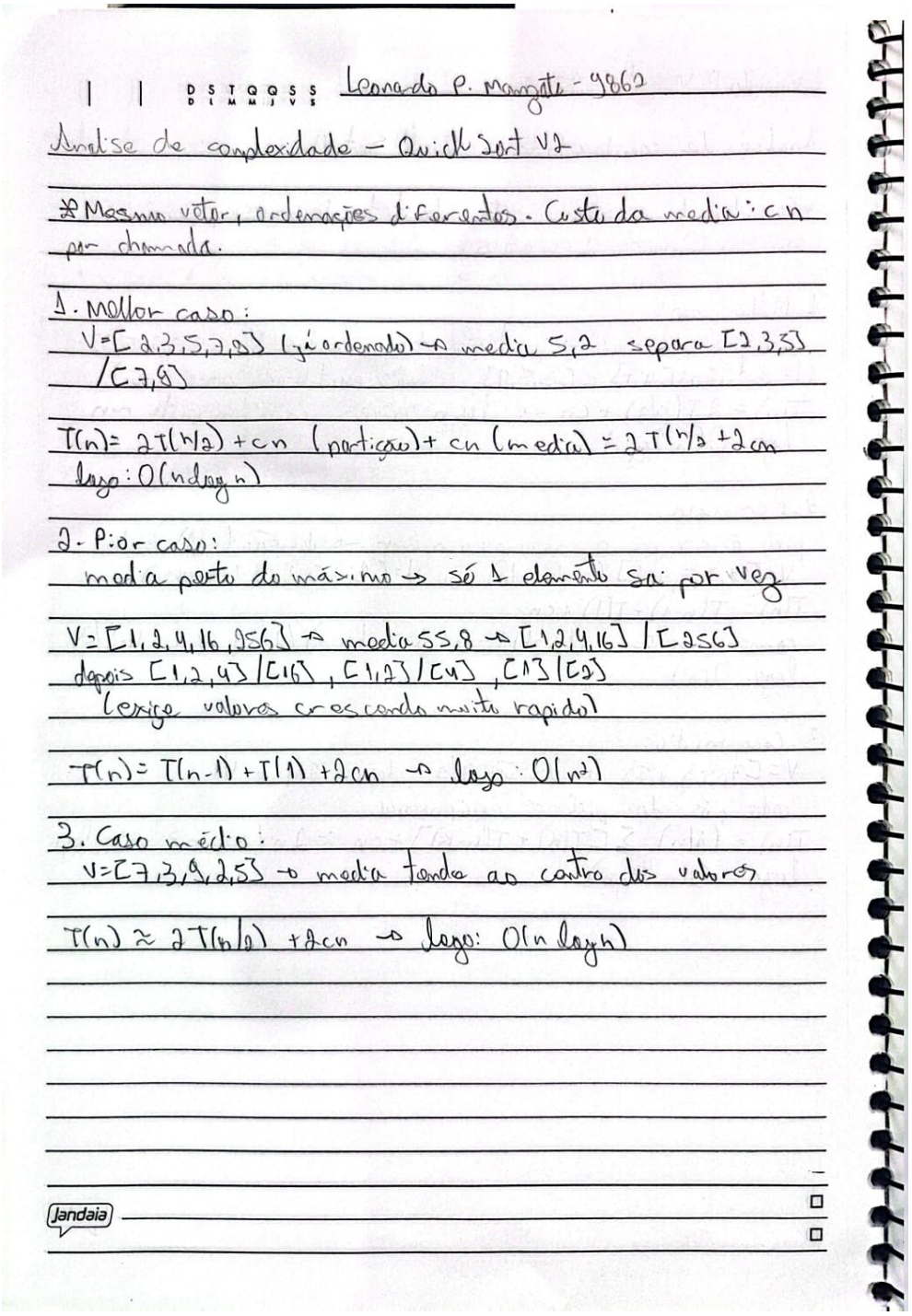


Figura 31: Análise de complexidade do algoritmo Quick Sort (versão 2, pivô pela média aritmética)

8 CONCLUSÃO

A realização deste trabalho permitiu compreender o funcionamento dos algoritmos *Insertion Sort*, *Bubble Sort*, *Selection Sort*, *Shell Sort*, *Merge Sort* e *Quick Sort*, este último em duas versões, além de analisar seus comportamentos em diferentes tipos de entrada. Por meio das implementações, dos testes de mesa, das análises de complexidade e dos experimentos práticos, foi possível observar as diferenças entre as estratégias utilizadas por cada algoritmo.

A análise teórica mostrou que o *Insertion Sort* apresenta complexidade $\Theta(n)$ no melhor caso, quando o vetor já está ordenado, e $\Theta(n^2)$ no caso médio e no pior caso. O *Bubble Sort* e o *Selection Sort*, considerando as implementações utilizadas neste trabalho, apresentam complexidade $\Theta(n^2)$ nos três casos analisados. Já o *Shell Sort* possui comportamento dependente da sequência de *gaps* utilizada. Na implementação estudada, com a divisão sucessiva do *gap* por 2, o melhor caso apresenta crescimento de ordem $\Theta(n \log n)$, enquanto o pior caso apresenta complexidade $\Theta(n^2)$. O *Merge Sort* apresenta complexidade temporal $\Theta(n \log n)$ no melhor caso, no caso médio e no pior caso, além de necessitar de memória auxiliar proporcional ao tamanho do vetor. Por fim, o *Quick Sort* apresenta complexidade $\Theta(n \log n)$ no melhor caso e no caso médio, e $\Theta(n^2)$ no pior caso, que ocorre quando o pivô gera partições muito desbalanceadas. Por ordenar o vetor no próprio local e realizar a chamada recursiva apenas sobre a menor partição, a implementação utilizada exige memória auxiliar de ordem $\Theta(\log n)$.

Os experimentos realizados com entradas crescentes, decrescentes e aleatórias permitiram observar na prática as diferenças previstas pela análise teórica. No *Insertion Sort*, a organização inicial dos elementos possui influência significativa no tempo de execução: o tempo é praticamente nulo quando o vetor já está ordenado e aproximadamente o dobro do observado em entradas aleatórias quando ele está em ordem decrescente. No *Selection Sort*, o tempo foi semelhante nos três tipos de entrada, pois a quantidade de comparações não depende da organização inicial. O *Bubble Sort* apresentou os maiores tempos em entradas decrescentes e aleatórias, mas, devido à interrupção antecipada quando não há trocas, teve tempo desprezível em entradas crescentes. O *Shell Sort* e o *Merge Sort* apresentaram tempos muito inferiores aos dos algoritmos quadráticos nos três tipos de entrada, o que evidencia a vantagem dos algoritmos de custo próximo de $\Theta(n \log n)$ para instâncias grandes.

Os experimentos com o *Quick Sort* evidenciaram a importância da escolha do pivô. A versão 1, que utiliza o primeiro elemento, apresentou tempos próximos aos dos algoritmos quadráticos em entradas crescentes e decrescentes, pois nesses casos o pivô é sempre o menor ou o maior valor da partição, levando o algoritmo ao pior caso. Em entradas aleatórias, porém, seu tempo foi comparável ao dos algoritmos de custo $\Theta(n \log n)$. A versão 2, que utiliza a média aritmética dos elementos como pivô, manteve as partições equilibradas nos três tipos de entrada, com tempos muito baixos, ao custo de um percurso adicional a cada particionamento. Vale ressaltar que esse bom desempenho depende da distribuição dos valores utilizados nos testes, pois a média aritmética pode gerar partições desbalanceadas quando os valores são muito discrepantes.

Também foi possível observar que o aumento do tamanho da entrada provoca um crescimento no tempo de execução dos algoritmos. Esse crescimento se torna mais significativo nos algoritmos e situações em que há maior quantidade de comparações, trocas ou deslocamentos. Dessa forma, os resultados experimentais possibilitam relacionar o comportamento observado nos gráficos e tabelas com as complexidades obtidas na análise teórica.

No caso do *Shell Sort*, teoricamente, a escolha do divisor influencia a sequência de *gaps* utilizada: um divisor menor produz uma quantidade maior de etapas, enquanto um divisor maior reduz essa quantidade. Entretanto, a quantidade de etapas, por si só, não determina o tempo de execução, pois a sequência de *gaps* também influencia a quantidade de elementos que precisam ser comparados e deslocados. Como a alteração do divisor não pôde ser implementada, os experimentos deste trabalho utilizaram apenas o divisor 2, e a comparação entre diferentes sequências de *gaps* permanece como possibilidade de trabalho futuro.

Dessa forma, o trabalho permitiu relacionar conceitos teóricos de análise de algoritmos com resultados obtidos por meio de experimentos práticos. A comparação entre os seis algoritmos, considerando as duas versões do *Quick Sort*, mostrou que, apesar de todos possuírem o mesmo objetivo de ordenar os elementos, suas estratégias de execução produzem comportamentos diferentes de acordo com o tamanho e a organização inicial da entrada. Os algoritmos de custo $\Theta(n \log n)$ se destacaram em instâncias grandes, desde que, no caso do *Quick Sort*, a escolha do pivô evite partições desbalanceadas.

9 REFERÊNCIAS BIBLIOGRÁFICAS

Referências

[1] DEVFURIA. *Introdução ao algoritmo de ordenação Insertion Sort*. Disponível em: http://devfuria.com.br/logica-de-programacao/introducao-ao-algoritmo-de-ordenacao-insertion-sort/. Acesso em: 14 set. 2026.

[2] CPLUSPLUS.COM. *rand – C++ Reference*. Disponível em: https://cplusplus.com/reference/cstdlib/rand/. Acesso em: 14 set. 2026.

[3] GURU99. *C++ Manipulação de arquivos: como abrir, gravar, ler e fechar arquivos em C++*. Disponível em: https://www.guru99.com/pt/cpp-file-read-write-open.html. Acesso em: 14 set. 2026.

[4] NGHIATU. *8 maneiras de medir o tempo de execução em C / C++*. Disponível em: https://ichi.nghiatu.com/pt/8-maneiras-de-medir-o-tempo-de-execucao-em-c-c-79591185633529. Acesso em: 14 set. 2026.

[5] STACK OVERFLOW EM PORTUGUÊS. *Como criar parâmetros opcionais em C++?*. Disponível em: https://pt.stackoverflow.com/questions/514913/como-criar-par%c3%a2metros-opcionais-em-c. Acesso em: 27 set. 2026.

[6] SCRIBD. *Método de Shell Sort Explicado*. Disponível em: https://pt.scribd.com/document/945722840/Shell-Sort. Acesso em: 27 set. 2026.

[7] TRYBE. *Bubble Sort: o que é e como usar? Exemplos práticos!*. Disponível em: https://blog.betrybe.com/tecnologia/bubble-sort-tudo-sobre/. Acesso em: 27 set. 2026.

[8] JOÃO ARTHUR BARBOSA MOURA. *Estruturas de Dados e Algoritmos*. Disponível em: https://joaoarthurbm.github.io/eda/posts/selection-sort/. Acesso em: 27 set. 2026.

[9] GEEKSFORGEEKS. *Comparison among Bubble Sort, Selection Sort and Insertion Sort*. Disponível em: https://www.geeksforgeeks.org/dsa/comparison-among-bubble-sort-selection-sort-and-insertion-sort/. Acesso em: 27 set. 2026.

[10] EDUCBA. *Selection sort vs Bubble sort | Differences of Selection sort vs Bubble sort*. Disponível em: https://www.educba.com/selection-sort-vs-bubble-sort/. Acesso em: 27 set. 2026.

[11] STACK OVERFLOW EM PORTUGUÊS. *Como passar uma função como parâmetro em C?*. Disponível em: https://pt.stackoverflow.com/questions/2983/como-passar-uma-fun%C3%A7%C3%A3o-como-par%C3%A2metro-em-c. Acesso em: 14 set. 2026.

[12] VISUALGO. *Sorting*. Disponível em: https://visualgo.net/en/sorting. Acesso em: 27 set. 2026.

[13] PROGRAMIZ. *Sorting Algorithms*. Disponível em: https://www.programiz.com/dsa/sorting-algorithm. Acesso em: 27 set. 2026.

[14] PROGRAMIZ. *Shell Sort*. Disponível em: https://www.programiz.com/dsa/shell-sort. Acesso em: 27 set. 2026.

[15] GIOVANI DA CRUZ. *Merge Sort: um método eficaz de ordenação*. Disponível em: https://giovanidacruz.com.br/merge-sort-um-metodo-eficaz-de-ordenacao/. Acesso em: 24 out. 2026.

[16] GEEKSFORGEEKS. *Merge Sort*. Disponível em: https://www.geeksforgeeks.org/dsa/merge-sort/. Acesso em: 24 out. 2026.

[17] CODDY. *Merge Sort — Visualização de algoritmo de ordenação*. Disponível em: https://coddy.tech/visualize/pt/sorting/merge-sort. Acesso em: 24 out. 2026.

[18] GEEKSFORGEEKS. *Quick Sort*. Disponível em: https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/. Acesso em: 9 nov. 2026.

[19] FEOFILOFF, Paulo. *Algoritmo Quicksort: análise da correção e do desempenho.* Instituto de Matemática e Estatística, Universidade de São Paulo. Disponível em: `https://users.ime.usp.br/~pf/algoritmos/aulas/quick.html`. Acesso em: 8 nov. 2026.

[20] PROGRAMIZ. *QuickSort (With Code in Python/C++/Java/C).* Disponível em: `https://www.programiz.com/dsa/quick-sort`. Acesso em: 9 nov. 2026.

[21] OPENAI. *ChatGPT*: ferramenta de inteligência artificial utilizada como auxílio na elaboração e revisão dos scripts de geração de gráficos e tabelas e organização LaTex no OverLeaf na elaboração do relatório. 2026.

10 Funções

10.1 Calculo do tempo de execução

Listing 7: Função utilizada para medir o tempo de execução

```
1  double calcularTempo(void (*algoritmo)(int*, int), int* vetor, int
       tamanho) {
2      // salvar tempo do inicio
3      auto inicio = chrono::steady_clock::now();
4
5      // executar o algoritmo
6      algoritmo(vetor, tamanho);
7
8      // salvar o tempo do fim da execucao
9      auto fim = chrono::steady_clock::now();
10
11     // calcular duracao
12     chrono::duration<double> tempo = fim - inicio;
13
14     return tempo.count();
15 }
```

A função apresentada no Código 7 é responsável por medir o tempo de execução do algoritmo. Para isso, registra o instante inicial, executa o algoritmo, registra o instante final e calcula a diferença entre os dois instantes.

O valor retornado pela função representa o tempo de execução em segundos.

10.2 Execução dos algoritmos

Listing 8: Função utilizada para executar os algoritmos de ordenação

```
void executarAlgoritmo(function<void(int*, int)> algoritmo,
                       const string& nomeAlgoritmo,
                       const string& caminhoBase,
                       const string& caminhoCSV) {

    int tipo = escolherTipoInstancia();
    int tamanho = escolherTamanhoInstancia();

    cout << "\nGerando instancia...\n";
    int* vetor = gerarInstancia(tipo, tamanho);

    if (vetor == nullptr) {
        cout << "Erro ao gerar instancia.\n";
        return;
    }

    const string nomeInstancia = obterNomeInstancia(tipo);
    const string nomeArquivo =
        nomeInstancia + "_" + to_string(tamanho) + ".txt";

    const string caminhoEntradaBase =
        caminhoBase + "Arquivos de Entrada/";
    const string caminhoSaidaBase =
        caminhoBase + "Arquivos de Saida/";
    const string caminhoTempoBase =
        caminhoBase + "Arquivos de Tempo/";

    criarDiretorios(caminhoEntradaBase, caminhoSaidaBase,
                    caminhoTempoBase, nomeInstancia);

    const string caminhoEntrada =
        caminhoEntradaBase + nomeInstancia + "/" + nomeArquivo;
```

```
33      const string caminhoSaida =
34          caminhoSaidaBase + nomeInstancia + "/" + nomeArquivo;
35      const string caminhoTempo =
36          caminhoTempoBase + nomeInstancia + "/" + nomeArquivo;
37
38      cout << "Salvando instancia de Arquivos de Entrada...\n";
39      salvarVetor(caminhoEntrada, vetor, tamanho);
40
41      cout << "Executando " << nomeAlgoritmo << "...\n";
42      double tempo = calcularTempo(algoritmo, vetor, tamanho);
43
44      cout << "Salvando instancia de Arquivos de Saida...\n";
45      salvarVetor(caminhoSaida, vetor, tamanho);
46
47      cout << "Salvando Arquivos de Tempo...\n";
48      salvarTempo(caminhoTempo, tamanho, tempo);
49      salvarCSV(caminhoCSV, tamanho, nomeInstancia, tempo);
50
51      cout << "\n===== EXECUCAO CONCLUIDA =====\n";
52      cout << "Algoritmo: " << nomeAlgoritmo << "\n";
53      cout << "Tipo: " << nomeInstancia << "\n";
54      cout << "Tamanho: " << tamanho << "\n";
55      cout << "Tempo: " << tempo << " segundos\n";
56
57      cout << "\nArquivos gerados:\n";
58      cout << "Entrada: " << caminhoEntrada << "\n";
59      cout << "Saida:   " << caminhoSaida << "\n";
60      cout << "Tempo:   " << caminhoTempo << "\n";
61      cout << "CSV:     " << caminhoCSV << "\n";
62
63      delete[] vetor;
64  }
```

A função apresentada no Código 8 é responsável por coordenar a execução dos algoritmos de ordenação. Inicialmente, solicita ao usuário o tipo e o tamanho da instância e

gera o vetor correspondente. Em seguida, define os caminhos dos arquivos e salva o vetor original no diretório de entrada.

Após essa etapa, a função executa o algoritmo recebido como parâmetro e mede seu tempo de execução por meio da função `calcularTempo`. Como o vetor é ordenado durante essa operação, seu conteúdo é salvo no diretório de saída. O tempo obtido também é armazenado em um arquivo individual e em um arquivo CSV, utilizado posteriormente para gerar tabelas e gráficos comparativos.

Por fim, são exibidas informações sobre a execução e os caminhos dos arquivos gerados. A memória alocada dinamicamente para armazenar o vetor é liberada com `delete[]`, evitando que essa alocação permaneça sem liberação.

Observação sobre o uso de funções lambda: Nas chamadas do Shell Sort, Merge Sort e do Quick Sort são utilizadas funções lambda para adaptar a chamada dos algoritmos à assinatura esperada pela função `executarAlgoritmo`, que recebe um ponteiro para função com dois parâmetros: o vetor e seu tamanho.

No Shell Sort, a função lambda permite passar o divisor do *gap* como argumento adicional para `shellSort`. O exemplo a seguir mostra como capturar a variável `divisor` para utilizar o valor informado pelo usuário:

Listing 9: Chamada do Shell Sort utilizando uma função lambda

```
executarAlgoritmo(
    [divisor](int* vetor, int tamanho) {
        shellSort(vetor, tamanho, divisor);
    },
    "Shell Sort",
    CAMINHO_SHELL,
    CSV_SHELL
);
```

Nesse exemplo, a captura `[divisor]` permite que a função lambda utilize o valor da variável externa `divisor`. É importante observar que, na implementação original, a chamada utiliza o valor fixo `2`. Para que o divisor informado pelo usuário seja utilizado, é necessário substituí-lo pela versão apresentada.

No Merge Sort, a função lambda permite chamar `mergeSort` passando apenas o vetor e seu tamanho, utilizando os valores padrão dos parâmetros `esq` e `dir`, conforme

apresentado no exemplo a seguir:

Listing 10: Chamada do Merge Sort utilizando uma função lambda

```
executarAlgoritmo(
    [](int* vetor, int tamanho) {
        mergeSort(vetor, tamanho);
    },
    "Merge Sort",
    CAMINHO_MERGE,
    CSV_MERGE
);
```

Nesse caso, os colchetes vazios `[]` indicam que a função lambda não precisa capturar variáveis externas. Ela recebe o vetor e seu tamanho e chama `mergeSort` com esses dois argumentos. Os parâmetros `esq` e `dir` assumem seus valores padrão, permitindo que a ordenação seja iniciada sobre o vetor inteiro.

Dessa forma, as funções lambda permitem que os dois algoritmos sejam executados pela mesma função `executarAlgoritmo`, mantendo a estrutura de geração das instâncias, medição do tempo e salvamento dos arquivos.